

Finite Element approximation of the Elliptic Eigenproblems: Convergence Analysis and Numerical Experiments

Aleksei Kuznetsov

I. INTRODUCTION

In modern science to solve complex problems various simulations based on mathematical models are used. By doing so, researchers are able to quickly test hypotheses, change parameters and solve equations that are too complex or impossible to physically recreate. One such numerical method is Finite Element Method (FEM) that is widely used to obtain approximate solutions of differential equations.

Many physical systems are governed by partial differential equations whose analytical solutions are either extremely difficult or impossible to obtain, that is the reason why numerical analysis comes in. It allows us to design and implement an algorithms to find approximate but highly accurate numeric solutions.

The finite element approximation has become one of the most widely used techniques for solving eigenvalue problems with elliptic boundary-value and eigenvalue problems. Its mathematical foundations have been extensively developed in classical texts such as Larson & Bengzon and Brenner & Scott. Numerous implementations have demonstrated that this numerical method provides accurate approximations whose errors decrease predictably under mesh refinement.

Hence, this report develops mathematical foundations of the finite element approximation of elliptic eigenvalue problems before implementing the method in Python for both Laplace and time-independent Schrödinger equations. Numerical experiments are performed using P1 and P2 finite elements and the convergence of eigenvalues and eigenfunctions is compared with theoretical predictions.

II. MATHEMATICAL BACKGROUND

In this section there would be given a brief introduction to the mathematical knowledge required to understand this project.

A. Elliptic partial differential equations

One of the branches of differential equations is elliptic differential equation, which has the general form [1]:

$$A \cdot u_{xx} + B \cdot u_{xy} + C \cdot u_{yy} + D \cdot u_x + E \cdot u_y + F \cdot u = G \quad (1)$$

The equation 1 is classified as elliptic if the discriminant is strictly negative [1]:

$$B^2 - 4AC < 0 \quad (2)$$

Elliptic equations commonly arise in steady-state or equilibrium problems. They generally describe equilibrium/spatial problems, while in comparison parabolic and hyperbolic equations are the classes associated with time evolution. Moreover, their solutions depend only on spatial variables and require boundary conditions rather than initial conditions, the most common are Dirichlet (specifying the value on the boundary) and Neumann (specifying the derivative or the flux at the boundary). In addition to this, changes at any point of the domain influences the solution, reflecting the global nature of elliptic problems.

B. Eigenvalue problems

Mathematically, an eigenvalue problem is an equation in which a linear transformation applied to a specific vector - eigenvector, results in the exact same vector scaled by a constant multiplier - eigenvalue. The basic eigenvalue problem for a linear transformation A acting on a nonzero vector x is defined as:

$$A\vec{x} = \lambda\vec{x} \quad (3)$$

where A is an $n \times n$ matrix or linear operator which is known; x is an eigenvector - a nonzero column vector whose direction is unchanged by the linear transformation, which need to be found; λ is an eigenvalue - a scalar that needs to be found.

The main idea is that when the transformation A operates on x , the eigenvector does not change its direction, it only stretches, compresses, or flips.

C. Generalised matrix eigenvalue problem

Another variation of the eigenvalue problem is the generalised eigenvalue problem. It can be represented in two forms:

- the vector form:

$$A\vec{x}_i = \lambda_i B\vec{x}_i \quad (4)$$

- the matrix form:

$$AX = BXD \quad (5)$$

Where X is the column matrix of the eigenvectors; D is a diagonal matrix of eigenvalues; B acts as a weighting (mass) matrix that defines a new inner product for the vector space. The main condition is that B is symmetric positive definite matrix.

In other words, it changes the geometry of the problem by measuring vectors not in the standard Euclidean space, but

relative to the properties of B itself [2]. It usually represents a specific geometric or physical constraint [3].

In addition, because both (in the Laplace case) matrices are symmetric, the solvers preserve these structural properties, guaranteeing real eigenvalues. Also, solving generalised problem avoids explicitly forming $B^{-1}A$, preserving symmetry and improving numerical accuracy, which would be useful for both Laplace and Schrödinger problems.

D. Weak formulation

Weak formulation is an approach of multiplying the differential equation by an arbitrary test function that will disappear at the end. More precisely it is an integral representation that relaxes the strict smoothness required for the strong form, therefore it requires the equation to be satisfied in an average or integral sense [5].

The way from the strong form to weak requires following steps:

Firstly, write the strong form (we will take Poisson equation on the interval I with Dirichlet boundary condition as an example) [6]:

$$\begin{cases} -u''(x) = f(x), & \text{where } x \in I = [0, L] \\ u(0) = u(L) = 0 \end{cases} \quad (6)$$

Then, multiply 6 by an arbitrary test function v (rewrite as variational formulation):

$$-u''(x)v(x) = f(x)v(x) \quad (7)$$

Where $v(x)$ is a continuous test function which is assumed to vanish at end points.

Finally, integrate over the domain using integration by parts (in our case the domain is the interval $I = [0, L]$) [7]:

$$\begin{aligned} \int_0^L f(x) \cdot v(x) dx &= \int_0^L -u''(x) \cdot v(x) dx \\ &= [-v(x) \cdot u'(x)] \Big|_0^L \\ &\quad + \int_0^L u'(x) \cdot v'(x) dx \\ &= \int_0^L u'(x) \cdot v'(x) dx \\ &\quad - u'(L) \cdot v(L) + u'(0) \cdot v(0) \end{aligned} \quad (8)$$

Since $v(0) = v(L) = 0$ by Dirichlet BCs:

$$\int_0^L f(x) \cdot v(x) dx = \int_0^L u'(x) \cdot v'(x) dx \quad (9)$$

The form obtained in 9 is called a weak form in 1D. For 2D and higher dimensions integration by parts follows the divergence theorem is used, in particular the Green's First Identity presented in 10 [7]:

$$\int_{\Omega} -\Delta uv dx = \int_{\Omega} \nabla u \cdot \nabla v dx - \int_{\partial\Omega} n \cdot \nabla uv ds \quad (10)$$

We are going from strong form to the weak one because weak form allows us to solve problems where derivatives

might be discontinuous or non-existent and it handles the boundary conditions naturally. In addition to this, it reduces the order of derivatives, which can simplify further computations.

E. Bilinear forms

In partial differential equations, a bilinear form is a mapping function $B : V \times V \rightarrow \mathbb{R}$ that takes two functions from a vector space and returns a scalar. In the weak formulation of a PDE, it represents the differential operator and boundary conditions. It is also linear in both arguments independently [9]. This is illustrated in 11 and 12 [10].

$$B(u + \alpha v, w) = B(u, w) + \alpha B(v, w) \quad (11)$$

$$B(u, v + \alpha w) = B(u, v) + \alpha B(u, w) \quad (12)$$

Therefore, the weak formulation of the Poisson equation 9 can be written in abstract variational form - find $u \in V$ such that it will satisfy the 13.

$$\begin{aligned} B(u, v) &= L(v) \quad \forall v \in V, \text{ where} \\ B(u, v) &= \int_{\Omega} \nabla u \cdot \nabla v dx \\ L(v) &= \int_{\Omega} f v dx \end{aligned} \quad (13)$$

F. Sobolev and Hilbert spaces

The main challenge for the numerical analysis is accepting that real world problems, discrete data sets and constraints fundamentally lack smoothness. Of course it is possible to force artificial smoothness, but it will lead to inefficiency, information loss or incorrect interpretation. That is why, in numerical methods scientists do not use functions from $C^n(\Omega)$ space, because requiring continuous derivatives is too restrictive for approximate solutions. That is where the Sobolev and Hilbert spaces come in.

Finite Element Method (FEM) uses piecewise-defined basis functions (this topic would be discussed later). Within each element the function might be perfectly smooth, but across element boundaries, the derivative is discontinuous. Therefore, we need to use the space that will allow us to have discontinuous derivatives, which is Sobolev space, however to be able to introduce the Sobolev space fully we need first to introduce the Lebesgue function space.

Lebesgue function space is a space of "p-th power integrable" functions which are defined by 14.

$$\begin{aligned} f \in L^p &\text{ if } \int |f|^p < \infty \text{ or} \\ &\int_{\Omega} |f(x)|^p dx < \infty \end{aligned} \quad (14)$$

And their norms or lengths are defined in 15:

$$\|f\|_{L^p} = \left(\int_{\Omega} |f|^p dx \right)^{\frac{1}{p}} \quad (15)$$

When $p = 2$, $L^2(\Omega)$ space possesses an inner product, allowing us to define angles, distances and perform orthogonal projections.

The Sobolev space, which is typically denoted as $W^{k,p}(\Omega)$, consists of all functions in $L^p(\Omega)$ whose weak (or distributional) derivatives up to order k also belong to $L^p(\Omega)$. The special case $W^{k,2}(\Omega)$, denoted by $H^k(\Omega)$ also known as Hilbert space, that is equipped with the corresponding inner product. Hilbert's space norm is presented in 16 and it is clear that it uses the L^2 norms.

$$\|u\|_{H^1} = \sqrt{\left(\int_{\Omega} |u|^2 dx + \int_{\Omega} |\nabla u|^2 dx \right)} \quad (16)$$

For solving PDEs we are going to use $H^1(\Omega)$ since it only requires first order derivatives which we have according to 9. Since we know that we have the Dirichlet boundary condition we can introduce $H_0^1(\Omega)$ that represents functions in the space $H^1(\Omega)$ whose boundary values are zero.

But why did we need all these spaces? The answer is that for the FEM if our functions will belong to the H^1 it requires the derivatives to be square integrable, but not continuous, therefore we will be able to integrate 9 over our domain full of piecewise polynomials.

G. Existence of the solution

Two of the most important questions in mathematics are the questions of existence and uniqueness. So what we want to ask while solving the PDE using FEM is why do we have a solution and is this solution unique. The answer for these questions is the Lax-Milgram theorem.

The Lax-Milgram theorem provides the uniqueness and existence of the solution of the variational problem. Formally, let V be a Hilbert space, $a : V \times V \rightarrow \mathbb{R}$ be continuous and coercive, and $L : V \rightarrow \mathbb{R}$ be a bounded linear functional. Then there exist a unique $u \in V$ such that 17 is true.

$$a(u, v) = L(v) \quad \forall v \in V \quad (17)$$

Where $\phi(x, x_f)$ is a bounded coercive bilinear form of a Hilbert space [11].

It is worth mentioning that Lax-Milgram theorem follows as a corollary to the Stampacchia theorems that states: any function $f \in H$ necessarily corresponds to a unique function $u \in H$ for which the inequality 18 is satisfied for all functions $v \in H$.

$$\phi(u, v - u) \geq \langle f, v - u \rangle_H \quad (18)$$

Where $\langle f, v - u \rangle_H$ is an inner product, which in Hilbert spaces defined by 19.

$$\langle u, v \rangle_{H^1(\Omega)} = \int_{\Omega} (uv + \nabla u \nabla v) dx \quad (19)$$

So using 13 we will have that $a(u, v)$ is a bilinear form and $L(v)$ is a linear functional.

Firstly, we need to say that $a(u, v)$ is bounded, in other words $|a(u, v)| \leq M \|u\| \|v\|$ (the proof can be found in [12]).

Then we show that our bilinear form is coercive (or elliptical) which is $a(v, v) \geq \alpha \|v\|_{H^1}^2$ for some $\alpha > 0$ (the proof can be found in A).

To establish continuity of $L(v)$, we must show that $|L(v)| \leq C_L \|v\|_{H^1}$ for a constant $C_L > 0$ (proof can be found in B).

Therefore, by the Lax-Milgram Theorem these properties guarantee that a unique exact solution $u \in V$ exists and stable. Moreover, the Lax-Milgram theorem establishes well-posedness for the corresponding source/Poisson variational problem. While the eigenvalue problem requires a different argument: we seek non-zero u and an eigenvalue λ satisfying the variational eigenproblem.

H. The Rayleigh-Ritz/Galerkin approximation

The idea explored in this section is the core of the Finite Element Method (FEM). The solution of PDEs belongs to infinite-dimensional space, so our goal is to go from the infinite-dimensional space to finite-dimensional subspace. To do so we will use two methods: Rayleigh-Ritz method and Galerkin discretisation which will give the equivalent solution of the variational problem [13].

We will now derive the Galerkin discretisation for the Laplace eigenvalue problem. We will look at the arbitrary problem $\lambda u = -\Delta u$, whose weak form can be expressed as in 20.

$$\lambda \int_{\Omega} uv dx = \int_{\Omega} \nabla u \nabla v dx \quad (20)$$

By definition of the weak form, the equation must hold for any arbitrary function v and it is in our power to put constraints on this test function. One of the main ones will be that our test space becomes discrete 21.

$$v_h \in V_h \quad \text{where } V_h = \text{span} \phi_1, \dots, \phi_n \subset V \quad (21)$$

Because our V_h is a finite-dimensional subspace of V , it is a closed subspace.

Therefore, if we require the weak formulation to hold for every $v_h \in V_h$ and because V_h is spanned by ϕ_j , it is sufficient to enforce it for any linear combination of them. So, we can rewrite 20 as it is shown in 22.

$$\lambda \int_{\Omega} u \phi_j dx = \int_{\Omega} \nabla u \nabla \phi_j dx \quad (22)$$

So, we can let $\phi_1, \dots, \phi_n$ be n arbitrary basis functions. Therefore, because we project our infinite-dimensional problem onto finite-dimensional subspace we can express our unknown function u as the linear combination of our basis functions ϕ_i and unknown coefficients c_i as it is shown in 23 [1] [7].

$$u_h = \sum_{i=1}^n c_i \phi_i \quad (23)$$

Substituting 23 into the 22 gives us 24.

$$\lambda \int_{\Omega} \sum_{i=1}^n c_i \phi_i \phi_j dx = \int_{\Omega} \nabla \left(\sum_{i=1}^n c_i \phi_i \right) \nabla \phi_j dx \quad (24)$$

Following the properties of integrals, we can rewrite 24 as 25.

$$\lambda \sum_{i=1}^n c_i \int_{\Omega} \phi_i \phi_j dx = \sum_{i=1}^n c_i \int_{\Omega} \nabla \phi_i \nabla \phi_j dx \quad (25)$$

This allows us to define 26.

$$\begin{aligned} A_{i,j} &= \int_{\Omega} \nabla \phi_i \nabla \phi_j dx \\ M_{i,j} &= \int_{\Omega} \phi_i \phi_j dx \\ \vec{c} &= (c_i)^T \end{aligned} \quad (26)$$

So, after putting 26 into 25 it becomes clear that $\sum_{i=1}^n A_{i,j}$ and $\sum_{i=1}^n M_{i,j}$ are the entries of the matrices, so we can rewrite 25 as 27.

$$\lambda M \vec{c} = A \vec{c} \quad (27)$$

Which is exactly the form of the generalised eigenvalue problem, where A is so-called stiffness matrix and M is a mass matrix (the properties and meaning of those matrices will be cleared later).

Galerkin discretisation is a process when we choose a finite-dimensional test/trial space and require the residual to be orthogonal to that space.

This is what makes FEA so powerful because it allows us to break down a complex continuous physical domain into a finite number of smaller, interconnected subregions (elements).

Since we know that it is difficult to explicitly calculate eigenvalues for a given domain $\Omega \subset \mathbb{R}^n$. Hence, from [14] we will show that eigenvalues are minimisers of a certain functional. To do so from 20 we can define 28 the Rayleigh Quotient for a given function w defined of the set $\Omega \subset \mathbb{R}^n$.

$$R(w) = \frac{\|\nabla w\|_{L^2(\Omega)}^2}{\|w\|_{L^2(\Omega)}^2} = \frac{\int_{\Omega} |\nabla w|^2 dx}{\int_{\Omega} |w|^2 dx} \quad (28)$$

The Rayleigh quotient can also be expressed in matrix form 29 which suits the discretised problem more.

$$R(c) = \frac{\vec{c}^T A \vec{c}}{\vec{c}^T M \vec{c}} \quad (29)$$

Now, we can introduce the Minimum Principle for the First Eigenvalue Theorem from [1] [14].

Let

$$Y \equiv w : w \in C^2(\Omega), w \not\equiv 0, w = 0 \text{ for } x \in \partial\Omega. \quad (30)$$

We call this the set of trial functions for $\lambda u = -\Delta u$. Suppose there exists a function $u \in Y$ such that u minimises the Rayleigh quotient over all trial functions $w \in Y$. That is,

$$m \equiv \frac{\|\nabla u\|^2}{\|u\|^2} = \min_{w \in Y} \left\{ \frac{\|\nabla w\|^2}{\|w\|^2} \right\} \quad (31)$$

Then m is the first eigenvalue of $\lambda u = -\Delta u$. That is, $m = \lambda_1$ and u is a corresponding eigenfunction.

The proof can be found in [14] page 3.

The Rayleigh-Ritz method restricts the Rayleigh quotient and minimisation problem to the finite-dimensional subspace.

Combining both Galerkin discretisation and Rayleigh-Ritz allows us to transform complex governing differential equations into a manageable system of algebraic equations that can be solved numerically.

The next chapter will describe what are the basis functions that we use to break the problem from infinite dimensions to finite, how they are defined and how to use them to solve elliptic PDEs.

III. FINITE ELEMENT METHOD (FEM)

Finite Element Method is a numerical method for approximating solutions of differential equations, particularly two-point boundary value problem and eigenvalue problem. The basic idea is to divide the domain into elements (in our case triangles) and look for a polynomial approximations to the unknown functions on each triangle, in other words we approximate locally - while the exact solution belongs to an appropriate function space, the FEM approximates the solution within each individual element using simple mathematical functions, typically linear or quadratic continuous piecewise polynomials. After that we rewrite the boundary value problem as a variational equation, and then seek a solution approximation to this equation from the space of continuous piecewise functions [7].

This method has few very important advantages. First of all, it can easily deal with complex geometries and higher order approximations of the solutions. Also, it works for many types of physics like structural loads, heat transfer, and fluid flow. Moreover, it can be easily visualised, which helps with finding the vulnerabilities in design [15].

In this project we will use 2D Finite Element Approximation to find solutions of the Laplace eigenproblem and the Time-Independent Schrödinger eigenproblem on the unit square domain.

A. Mesh

In order to divide our domain into smaller pieces we first need to introduce the pattern of how we are going to divide it.

1) *Triangulations*: If we want to divide the interval in 1D we will use intervals, that can be either equal to each other or be different length. In 2D idea is the same, but instead of small line intervals we need to add one more point to be able to move from 1D line to 2D surface which is a triangle. The domain divided into triangles is called triangulation.

We next will give the formal definition from [7].

Let $\Omega \subset \mathbb{R}^2$ be a bounded polygonal domain with boundary $\partial\Omega$. A triangulation, or a mesh $\mathcal{K}$ of Ω is a set $\{K\}$ of triangles K , such that $\Omega = \bigcup_{K \in \mathcal{K}} K$, and such that the intersection of two triangles is either an edge, a corner, or empty. No node is allowed to be hanging, that is lie on the edge of another triangle. The corners of the triangles are called the nodes [7].

2) *Mesh generation*: To generate the triangular mesh the built-in Delaunay algorithm was used [20]. Which is taking a set of 2D coordinates and determines the triangulations with the given points as triangle nodes. This method has

own advantages and disadvantages. Delaunay algorithm are optimal in the sense that the angles of all triangles are maximal, however practically when the mesh needs to respect the boundaries of the domain, it can lead to triangles with poor quality [7].

B. Piecewise polynomial spaces

In order to interpolate our unknown function on each element we need to decide what family of functions we are going to use for this interpolation. Polynomials are the best candidates for this because they are mathematically simple to integrate and differentiate, they can be easily translated into code and the accuracy will be easily improved by increasing the order of the polynomials. And because in this project we are going to use first order (P1) and second order polynomials (P2) to interpolate our unknown function on each element of the mesh that is why we "introduce" the space of piecewise polynomials.

1) *P1 Elements*: To construct the piecewise linear function, we are letting K be a triangle and in 32 we define the space of linear functions on K denotes $P_1(K)$.

$$P_1(K) = \{v : v = c_0 + c_1x_1 + c_2x_2, (x_1, x_2) \in K, c_0, c_1, c_2 \in \mathbb{R}\} \quad (32)$$

Simply, $P_1(K)$ contains all functions of the form $v = c_0 + c_1x_1 + c_2x_2$ on K .

We know from [7] that a function v in V_h is uniquely determined by its nodal values. So, we let the nodal values be our degrees of freedom construct the corresponding basis $\{\phi_j\}_{j=1}^{n_p} \subset V_h$ with the condition defined in 33, because these functions define the basis they are called basis functions.

$$\phi_j(N_i) = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad i, j = 1, 2, \dots, n_p \quad (33)$$

The Fig. 1 illustrates a typical basis function ϕ_j which because of their shape sometimes are called hat functions.

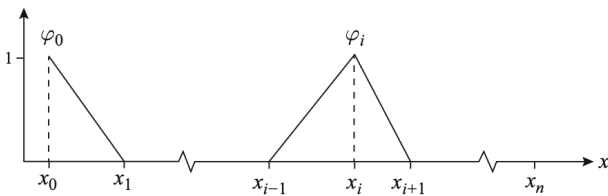


Fig. 1: Hat function [7]

From linear algebra we know that we can write every function $v \in V_h$ using our basis functions. This is shown in 34.

$$v = \sum_{i=1}^{n_p} \alpha_i \phi_i \quad (34)$$

Where $\alpha_i = v(N_i)$, $i = 1, 2, \dots, n_p$, are the nodal values of v .

It is worth mentioning since the project explores the convergence of the Finite Element Approximation that P1 elements have the theoretical convergence $O(h^2)$. This number comes from the [16] and [17] that is presented in 35.

$$|\lambda - \lambda_h| \leq Ch^{2p} \quad (35)$$

Therefore, when $p = 1$ the convergence of the error will be h^2 . This double convergence rate relative to the H^1 norm occurs because of the Rayleigh quotient property 28 [18]. Simply, the error in the eigenfunction is bounded like it is presented in 36.

$$\|u - u_h\|_{H^1(\Omega)} \leq Ch^p \quad (36)$$

The eigenvalue error acts quadratically relative to the eigenvector error [19], so we have:

$$|\lambda - \lambda_h| \leq C\|u - u_h\|_{H^1(\Omega)}^2 \leq C(h^p)^2 = Ch^{2p} \quad (37)$$

As a result, we are expecting to get this number in our computer implementation.

2) *P2 Elements*: P1 elements are easy to implement and give good interpolations, but as it was discussed earlier the main advantage of the polynomials is their faster convergence and more precision with increasing of the degree.

So, instead of the polynomial space given in 32, we are going to use the space of the second degree polynomials $P_2(K)$ defined in 38.

$$P_2(K) = \{v : v = c_0 + c_1x_1 + c_2x_2 + c_3x_1^2 + c_4x_1x_2 + c_5x_2^2, (x_1, x_2) \in K, c_0, \dots, c_5 \in \mathbb{R}\} \quad (38)$$

In Fig. 2 we can see how the basis functions will behave.

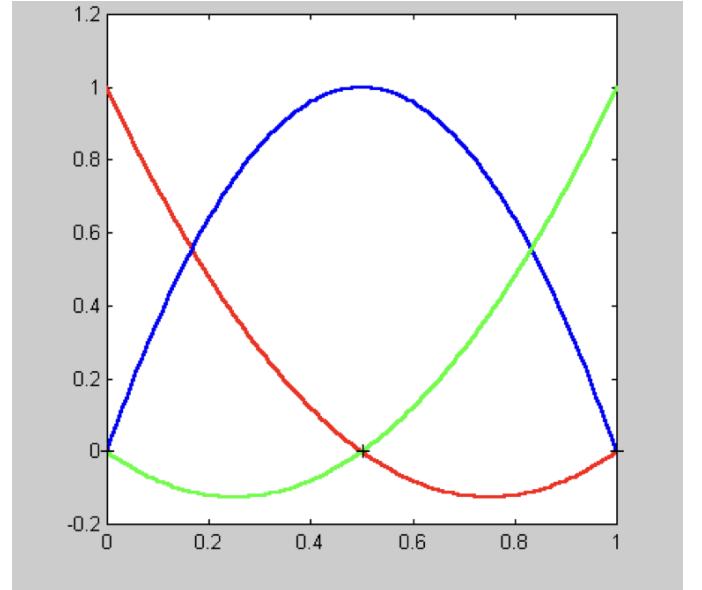


Fig. 2: P2 Hat function [23]

Since we have quadratic polynomials that means that the gradients will be linear within each element. This provides local flexibility for representing curved solutions, therefore,

quadratic polynomials will handle these complexities more naturally comparing to P1 elements.

For sufficiently regular eigenfunctions and an appropriate family of meshes, the eigenvalue error is expected to behave as 35 and give the convergence of $O(h^4)$, which is double comparing to the P1 elements. This can also be explained in less mathematical language - because higher order elements can achieve a given accuracy with fewer elements, as each element has more degrees of freedom.

C. Reference (Standard) element

Now, when know how to divide our domain into triangulations, and how to interpolate unknown function on each triangle using continuous piecewise polynomials, we will have a question: do we have to do this to each triangle individually? In practice this approach would be computationally inefficient because we can deal with the complex triangulations that will have different sizes, shapes and orientations, so interpolate each one individually will take a lot of time and effort. That is why we need to come up with the "common factor" that we can relate to every element of our mesh. That is why we are going to use reference triangle.

By definition reference triangle is a triangle with vertices in: $(1, 0)$, $(0, 1)$ and $(0, 0)$ [21]. In order to avoid confusion with the coordinates, we will say every triangle in (x, y) is mapped to the reference coordinate plane (ξ, η) , this mapping is illustrated in Fig. 3.

Any triangle with corners $P_i(x_i, y_i)$, $i = 1, 2, 3$ can be mapped from the reference element to physical as in 39, using the counterclockwise numbering.

$$\begin{aligned} x &= x_1 + (x_2 - x_1)\xi + (x_3 - x_1)\eta \\ y &= y_1 + (y_2 - y_1)\xi + (y_3 - y_1)\eta \end{aligned} \quad (39)$$

However, in order to map any function from physical coordinates we have to the transformation which is represented by Jacobian matrix 40.

$$J = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{bmatrix} \quad (40)$$

This matrix connects $dx dy$ to $d\xi d\eta$ and transforms gradients between reference and physical elements.

1) *Linear approximation*: The aim is to approximate the unknown function by a linear form of 41.

$$u(\xi, \eta) = c_1 + c_2\xi + c_3\eta \quad (41)$$

The triangle presented on the figure is also know as Linear Lagrange Triangle So from this we obtain the system 42.

$$\begin{aligned} u_1 &= u(0, 0) = c_1 \\ u_2 &= u(1, 0) = c_1 + c_2 \\ u_3 &= u(0, 1) = c_1 + c_3 \end{aligned} \quad (42)$$

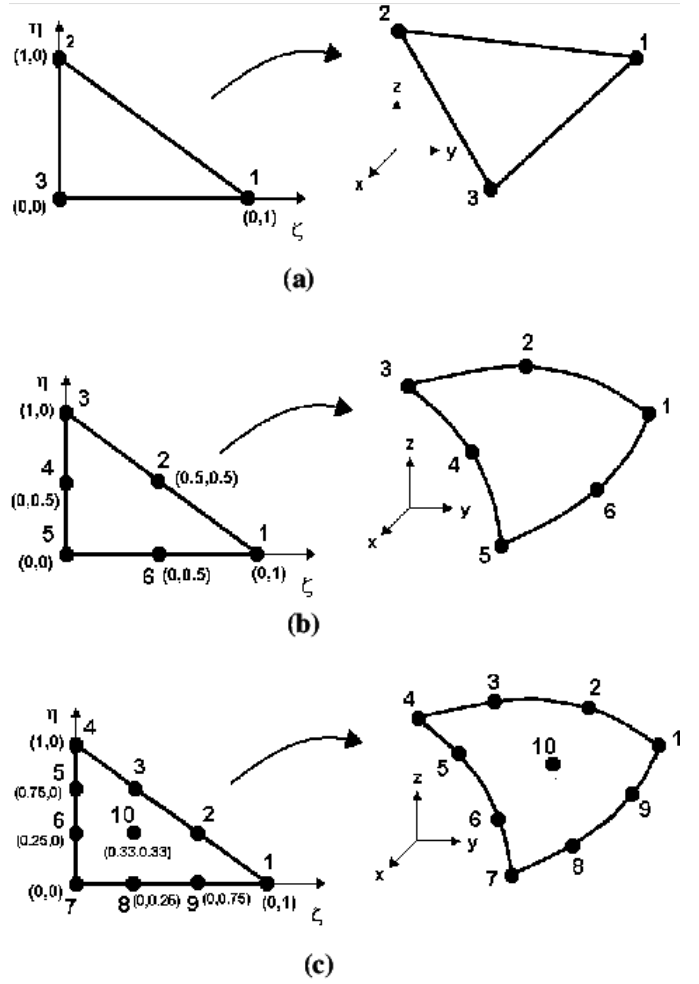


Fig. 3: Reference triangle mapping for the (a) linear, (b) quadratic and (c) cubic elements [22]

Which will give us the coefficients as a function of the values at the grid nodes by matrix inversion:

$$c = Au, \text{ where}$$

$$A = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \quad (43)$$

From matrix A we calculate the linear basis functions for triangles like it is presented in 44

$$\begin{aligned} N_1(\xi, \eta) &= 1 - \xi - \eta \\ N_2(\xi, \eta) &= \xi \\ N_3(\xi, \eta) &= \eta \end{aligned} \quad (44)$$

In Fig. 4 we can see how the linear approximation looks like.

2) *Quadratic approximation*: As it was mentioned before in P2 elements, we can approximate any function defined on the triangle by a quadratic function 38. So in the reference coordinates it will have the form 45.

$$u(\xi, \eta) = c_1 + c_2\xi + c_3\eta + c_4\xi^2 + c_5\xi\eta + c_6\eta^2 \quad (45)$$

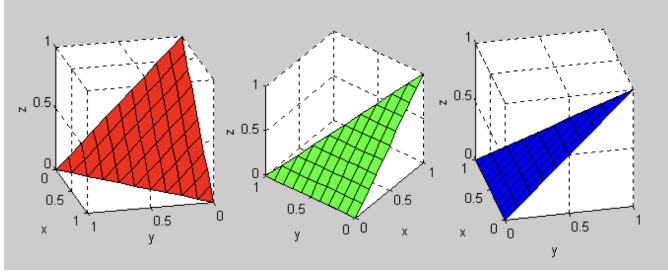


Fig. 4: Linear approximation [23]

However, since the quadratic functions require more points to be defined properly we need some additional points the new reference triangle is presented on the figure also known as Quadratic Lagrange Triangle.

Following the same procedure as in the III-C1 we compute the function u at each grid point.

$$\begin{aligned}
 u_1 &= c_1 \\
 u_2 &= c_1 + c_2 + c_4 \\
 u_3 &= c_1 + c_3 + c_6 \\
 u_4 &= c_1 + \frac{1}{2}c_2 + \frac{1}{4}c_4 \\
 u_5 &= c_1 + \frac{1}{2}c_2 + \frac{1}{2}c_3 + \frac{1}{4}c_4 + \frac{1}{4}c_5 + \frac{1}{4}c_6 \\
 u_6 &= c_1 + \frac{1}{2}c_3 + \frac{1}{4}c_6
 \end{aligned} \tag{46}$$

Following the matrix inversion we can compute the coefficients presented in 47

$$c = Au, \text{ where } A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -3 & -1 & 0 & 4 & 0 & 0 \\ -3 & 0 & -1 & 0 & 0 & 4 \\ 2 & 2 & 0 & -4 & 0 & 0 \\ 4 & 0 & 0 & -4 & 4 & -4 \\ 2 & 0 & 2 & 0 & 0 & -4 \end{bmatrix} \tag{47}$$

This allow us to obtain basis functions in 48 and illustrated in figure

$$\begin{aligned}
 N_1(\xi, \eta) &= (1 - \xi - \eta)(1 - 2\xi - 2\eta) \\
 N_2(\xi, \eta) &= \xi(2\xi - 1) \\
 N_3(\xi, \eta) &= \eta(2\eta - 1) \\
 N_4(\xi, \eta) &= 4\xi(1 - \xi - \eta) \\
 N_5(\xi, \eta) &= 4\xi\eta \\
 N_6(\xi, \eta) &= 4\eta(1 - \xi - \eta)
 \end{aligned} \tag{48}$$

As it is seen from the figure the this approximation is more complex but allows us to get more detailed picture about our unknown function. The shape of these basis functions is presented in Fig. 5.

Since we have more degrees of freedom this means that more functions will participate in the approximation of the same area.

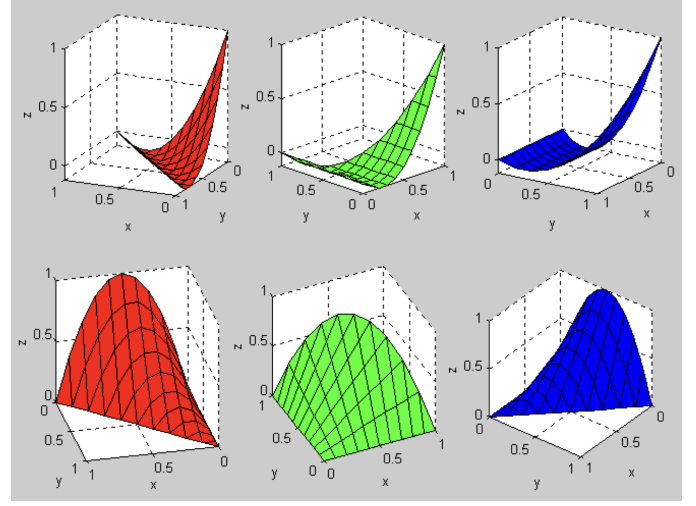


Fig. 5: Quadratic approximations [23]

D. Conclusion

From everything that was discussed previously we can build the general picture of the project. This can be described simply as we are given a differential equation, whose solution is unknown function. Since the solution can be any family of functions we restrict ourselves to piecewise polynomial functions. Therefore we can reduce the infinite-dimensional problem to finite-dimensional matrix eigenvalue problem, which we know how to solve from the course of linear algebra.

In the next chapter we will go from discussing abstract concepts to eigenproblems that have the physical meaning.

IV. ELLIPTIC EIGENPROBLEMS

Having all the mathematical background can be very interesting, however what is the practical application of everything that was discussed before?

In this section the two physical eigenproblems would be introduced to show the application of the finite element method.

A. The Laplace Eigenproblem

The Laplace eigenproblem is the partial differential equation on the form 49 on a domain with set of boundary conditions, in our case - Dirichlet.

$$\begin{cases} -\Delta u(x) = \lambda u(x), & x \in \Omega \\ u(x) = 0, & x \in \partial\Omega \text{ (Dirichlet BC)} \end{cases} \tag{49}$$

Here, Δ is the Laplacian operator that expands in 50, λ represents eigenvalues and u is the unknown eigenfunction.

$$\Delta = \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) \tag{50}$$

This problem has a physical meaning - the feedback mechanism of the physical system. It states that the spatial acceleration (curvature) of a state is directly proportional and opposite to the state itself.

For example, if we look at heat equation 51.

$$u_t - \Delta u = 0 \quad (51)$$

Then with separation of variables we are getting 52.

$$u(x, t) = \phi(x)e^{-\lambda t} \quad (52)$$

Substituting back to 51 and assigning $\lambda = e^{-\lambda t}$ gives us 53.

$$-\Delta \phi = \lambda \phi \quad (53)$$

Hence, from 53 it is clear that eigenvalues in this case determine the rate of fundamental thermal decay and the eigenfunction shows the elemental spatial distributions of heat.

Therefore, the smallest eigenvalue λ_1 corresponds to the slowest-decaying mode, in other words the most resilient thermal profile that takes the longest time to cool down to ambient temperature.

If we recall the derivation of the weak form we get the generalised Laplace eigenproblem 54.

$$Au = \lambda Mu \quad (54)$$

B. Schrödinger's Eigenproblem

Schrödinger's equation is one of the most important equations of the 20th century. The Schrödinger equation plays a fundamental dynamical role in non-relativistic quantum mechanics, analogues in importance to the role played by Newton's laws in classical mechanics. It tells us how waves of probability change and behave for simple particles over time, however in this project we will look at the Time-Independent Schrödinger's Equation 55.

$$\left[-\frac{\hbar^2}{2m} \nabla^2 + V(\vec{r}) \right] \psi(\vec{r}) = E\psi(\vec{r}) \quad (55)$$

where $\hbar$ - reduced Planck's constant; ψ - wave function, it describes the particle's quantum state; m - the mass of the studied particle; V - potential energy of the system.

In fact the left part of the equation is known as Hamiltonian operator 56.

$$\hat{H} = \left[-\frac{\hbar^2}{2m} \nabla^2 + V(\vec{r}) \right] \quad (56)$$

Therefore the 55 can be rewritten as 57.

$$\hat{H}\psi = E\psi \quad (57)$$

This problem is very crucial for physics however it is not different from the Laplace eigenproblem. If we take a look at a special case of Schrödinger equation called - the infinite square well, we will have the potential energy $V = 0$. Hence, 55 will take the form 58.

$$-\frac{\hbar^2}{2m} \Delta \psi = E\psi \quad (58)$$

If we rearrange the 58 and will call $\lambda = \frac{2mE}{\hbar^2}$ we will have:

$$-\Delta \psi = \lambda \psi \quad (59)$$

Which is exactly the form of the Laplace eigenproblem. Additionally, the infinite square well requires the infinite potential outside the domain, which is the Dirichlet boundary condition. Solving this problem will give us the wave function, whose square $|\psi(x)|^2$ represents the Born interpretation of probability density of finding a quantum particle at a specific position in space. We have to do that because according to the Heisenberg's Uncertainty Principle we cannot determine the particle's position and momentum at the same time.

Although the infinite square well is an idealised model, it provides a useful benchmark because its eigenvalues and eigenfunctions are known analytically.

Furthermore, 57 can be rewritten as a generalised eigenproblem 60. However, to do so we need to define $\lambda = \frac{2mE}{\hbar^2}$ and appropriately scale the potential.

$$(A + K_V)\psi = \lambda M\psi \quad (60)$$

Where $(K_V)_{ij} = \frac{2m}{\hbar^2} \int_{\Omega} V(x)\phi_i\phi_j dx$ is a potential matrix, with $V(x)$ the potential energy function.

From 60 we can see that mathematically the statement of the problem is very similar to the Laplace eigenproblem 54.

C. Conclusion

Looking at what was previously discussed in this section the idea behind this project becomes clear:

Firstly, we will take the simplest 2D eigenproblem - Laplace, will explore it, learn how to solve it numerically and interpret the results;

Secondly, we will increase the complexity by adding another operation of matrix addition - add potential matrix to the existing stiffness matrix. It should be clear that the code and the logic behind it should not change by going from Laplace to solve Schrödinger eigenproblem.

This project will show that problems that look completely different from the physical point of view are solved the same way, with different interpretation of the results.

Next sections will show the implementation part of this project.

V. GOALS AND OBJECTIVES

The idea behind this project is to research about solving 1D and 2D partial differential equations using Finite Element Method and implement 2D solutions in practice. Another goal is compare and analyse the convergence rates of P1 and P2 elements and compare them to the known data.

VI. IMPLEMENTATION

In order to complete this project, the Visual Studio code was used to write and run the code in Python and Jupiter Notebooks.

In order to simplify the work various libraries were used, like NumPy, Math and SciPy for different mathematical operations; Pandas for presenting results in good-looking tables; and Matplotlib and PyVista to visualise plots in 2D and 3D respectively.

A. Mesh generation

To generate mesh we will use the square with coordinates $[0, 1] \times [0, 1]$. On this square domain we will build triangulations using the Delaunay algorithm from [7]. In our example we will hard code the number of points per axis, but it is possible to let the user choose the number of points. The mesh with 6 points per axis is illustrated on the Fig. 6 for P1 elements and Fig. 7 for P2 elements. In order to avoid breaking the code all the triangulations were sorted in the ascending order.

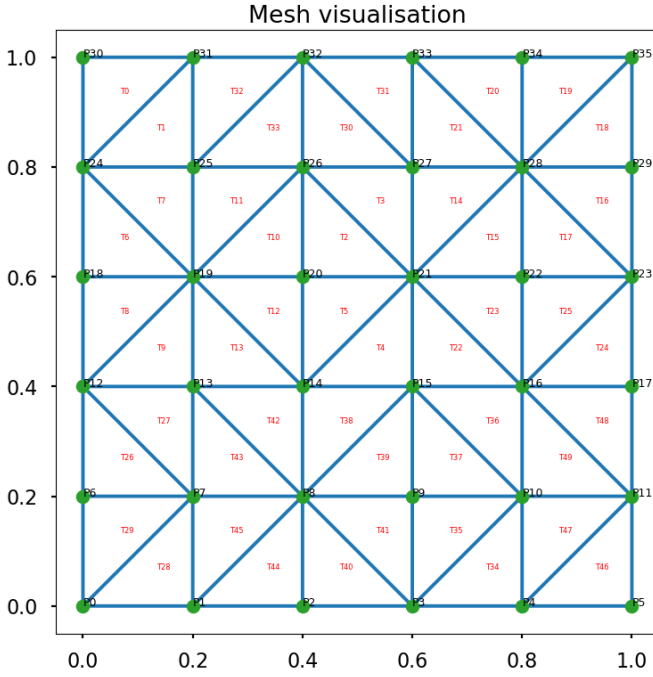


Fig. 6: Mesh with P1 triangles

B. Local element matrices

On every triangle, we compute local stiffness, mass and potential-energy matrices (for the Schrödinger problem). These local matrices describe the contribution of each element to the corresponding global matrices.

The local stiffness matrix A measures the alignment of the gradients of two basis functions or simpler how much the element resist deformation (spring force).

The local mass matrix M measures how much basis functions occupy the same region of space (overlap), in other words how much the triangle resists acceleration.

The local potential matrix K_V measures the potential energy contribution between basis functions [24].

Following [7] we know that each entry of stiffness matrix and mass matrix are calculated by 61 and 62 respectively.

$$(A_K)_{ij} = \int_K \nabla \phi_i \cdot \nabla \phi_j \, dxdy \quad (61)$$

$$(M_K)_{ij} = \int_K \phi_i \phi_j \, dxdy \quad (62)$$

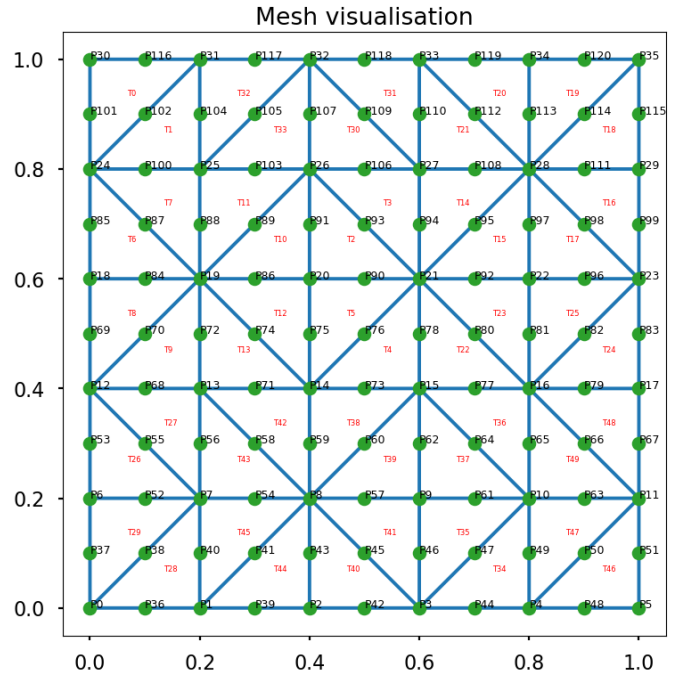


Fig. 7: Mesh with P2 triangles

With potential energy matrix $K_{V,K}$ the situation is a bit more complex because potential energy in Schrödinger Equation can be defined with various equations. But nevertheless we know that potential matrix $K_{V,K}$ is calculated using 63.

$$(K_{V,K})_{ij} = \frac{2m}{\hbar^2} \int_K V(x) \phi_i \phi_j \, dxdy \quad (63)$$

However, because evaluating the element integrals analytically for every element and potentially every potential function is inconvenient, we will use the Gaussian quadrature that will provide an efficient approximation.

To start with, we will use our reference mapping $(x, y) \mapsto (\xi, \eta)$. To do so we recall the change of variables.

$$\int_K f(x, y) \, dxdy = \int_{\hat{K}} f(\xi, \eta) |det(J(\xi, \eta))| \, d\xi d\eta \quad (64)$$

where $det(J(\xi, \eta))$ is the determinant of the Jacobian matrix defined in 40.

However, for the linear Lagrange finite element, that we are using, the Jacobian matrix is given by 65 [7].

$$J = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{bmatrix} \quad (65)$$

Therefore, the determinant of J is

$$det(J) = 2|K| \quad (66)$$

Now approximating the integral over $\hat{K}$ from 64 by a quadrature formula we get 67.

$$\int_{\hat{K}} f(\xi, \eta) |det(J(\xi, \eta))| \, d\xi d\eta \approx \sum_{q=1}^{n_q} \frac{w_q}{2} f(\xi_q, \eta_q) |det(J(\xi_q, \eta_q))| \quad (67)$$

where N_q is the number of quadrature points, w_q the quadrature weights, and (ξ_q, η_q) the quadrature points.

For P1 elements, the mass integrand $\phi_i \phi_j$ has polynomial degree 2, while the stiffness integrand $\nabla \phi_i \cdot \nabla \phi_j$ is constant. For P2 elements, these degrees become 4 and 2, respectively. Therefore, a quadrature rule exact for polynomials of degree at least 4 is sufficient to evaluate the Laplace element integrals exactly in exact arithmetic. That is why we will use the 6 point Hammer-Marlowe-Stroud quadrature rules [25]. This rule has more quadrature points then required for solving this type of integral, however it gives more precision with minor speed sacrifices.

However, for the Schrödinger problem, the degree of the potential integrand additionally depends on the chosen potential $V(x, y)$. For polynomial potentials, increasing the degree of V increases the degree of the integrand $V \phi_i \phi_j$, requiring a quadrature rule of sufficiently high degree. For example, a 12 point David Dunavant's quadrature rule [26] with Burkardt's implementation from [27].

An important property of the local matrices is their symmetry. This follows directly from the definitions. For example,

$$(A_K)_{ij} = \int_K \nabla \phi_i \cdot \nabla \phi_j \, dx dy \quad (68)$$

Since the dot product is commutative, $\nabla \phi_i \cdot \nabla \phi_j = \nabla \phi_j \cdot \nabla \phi_i$, therefore we have:

$$(A_K)_{ij} = (A_K)_{ji}. \quad (69)$$

The same argument applies to M_K and $K_{V,K}$, provided that the potential V is real-valued.

Thus, the resulting matrices should satisfy $A_K = A_K^T$, $M_K = M_K^T$, and $K_{V,K} = K_{V,K}^T$.

C. Global assembly

Once we obtained the local matrices that represent certain physical properties on every small element of the mesh we can build the picture back piece by piece.

To do so, we need to follow the straightforward algorithm called assembly [28]. Each local element matrix is expressed with respect to the local basis functions associated with the nodes of that element. These local degrees of freedom correspond to specific global degrees of freedom in the mesh. During assembly, each local matrix is added to the corresponding entries of the global matrix [7].

More detailed, if an element K has global node indices N_1 , N_2 and N_3 , its local matrix A_K contributes like it is shown in 70.

$$A_{N_i, N_j} += (A_K)_{ij} \quad i, j = 1, 2, 3 \quad (70)$$

The same procedure applies to local mass and potential matrices:

$$\begin{aligned} M_{N_i, N_j} &+= (M_K)_{ij} \\ (K_V)_{N_i, N_j} &+= (K_{V,K})_{ij} \end{aligned} \quad (71)$$

Since neighbouring elements share same nodes, several local matrices contribute to the same entries of the global

matrices. This assembly process produces the global matrices representing the entire computational domain.

The more detailed algorithm can be found in [28] or [29].

The algorithm may seem complicated, but in reality it is very straightforward, so to illustrate it the Fig. 8 presents more clear process of the assembly.

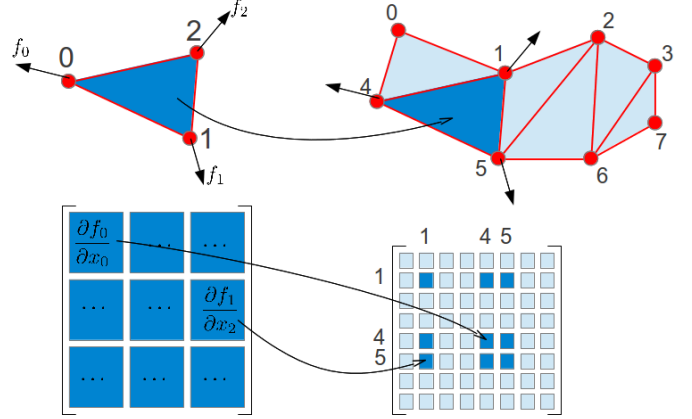


Fig. 8: Assembly process [30]

D. Dirichlet boundary conditions

The Dirichlet (or first-type) boundary condition is a type of boundary condition which specifies the values a solution needs to take on the boundary of the domain [31] in other words we prescribe the value of the solution at the boundary [7].

In this project we will take the simple special case - homogeneous Dirichlet boundary condition. We will make our function 0 on the boundary. What does it mean for our algorithm and assembly?

When we assemble our global matrices from local pieces, every point of the domain is used. Hence, since we know that our solution will take the zero values, we can simplify our global matrix and avoid extra calculations that will vanish at the end.

The basic idea is that we can consequently restrict the finite-element space to the subspace of functions whose boundary degrees of freedom vanish [32]. Algebraically, this corresponds to extracting the interior-interior blocks of the assembled matrices:

$$A_h = A_{II}, \quad M_h = M_{II} \quad (72)$$

The resulting reduced generalized eigenvalue problem is:

$$A_h u_h = \lambda M_h u_h \quad (73)$$

The reconstructed finite-element eigenfunction has zero coefficients at the boundary nodes and therefore satisfies the homogeneous Dirichlet condition [33].

E. Eigenvalue solver

In order to find eigenvalues and eigenvectors we use the SciPy library, in particular `scipy.linalg.eigh`. It solves a standard or generalised eigenvalue problem for a complex

Hermitian or real symmetric matrix [34]. From VI-B we know that all of our matrices are symmetric and real, that is why it would be appropriate to use it to find both eigenvalues and eigenvectors.

This method returns the 1D array of eigenvalues in ascending order and the 2D array of the normalised eigenvectors, where the column $v[:, i]$ is corresponding to the eigenvalue $w[i]$ [34].

VII. EIGENVALUES

The previous sections established the mathematical formulation and numerical implementation of the finite element eigenvalue problem. In this section, the obtained eigenvalues are analysed to compute the approximation error and to investigate the convergence behaviour of the P1 and P2 finite element approximations.

The `scipy.linalg.eigh` returned us the array of eigenvalues, but how are we going to check if the numbers are correct?

One of the ways is to make the calculations by hand and compare with obtained results (this was done for some eigenvalues on small mesh sizes), however it becomes more and more difficult to check every mesh refinement.

Another way is to compare results with the exact values. For the unit square $\Omega = (0, 1)^2$ with homogeneous Dirichlet boundary condition, the analytical eigenpairs can be obtained by separation of variables [16] [35]. For the Laplace eigenproblem the analytical eigenvalues can be obtained by 74.

$$\lambda_{m,n} = \pi^2(m^2 + n^2) \quad (74)$$

where $m, n = 1, 2, \dots$ [35].

It is clear that with $m = 1, n = 2$ and $m = 2, n = 1$ we will have the same eigenvalue, this is called degenerate eigenvalue [19] [35]. It happens, because of the underlying symmetry in the system which forces different states or modes to share the same eigenvalue like an energy or frequency [36]. For example, for the Laplace eigenproblem, we can say that distinct vibrational modes produce the exact same resonant frequency because swapping the x and y dimensions yields the same energy. For the Schrödinger eigenproblem, we can say that electrons in an atom can have different quantum states that share the same energy level due to rotational symmetry.

Hence, we will compute the exact eigenvalues and observe how close our approximations are.

But, in order to get the most from the analysis we will also use another method, that is applied in practice, when computing the exact values does not seem feasible. We will check how the eigenvalues converge with the mesh refinement. This method is useful on practice because we know the convergence of FEM, $O(h^2)$ for P1 elements and $O(h^4)$ for P2 elements, in other words For a sufficiently regular problem and an appropriate family of meshes, the eigenvalue error is expected to decrease asymptotically as $O(h^{2p})$ [19]. Thus, as the mesh is refined, the computed eigenvalues should approach the exact eigenvalues at the theoretically predicted rate. To do so we will use the power law. Power law is a relationship between the discretisation error E_h and characteristic mesh

size h . For a method with convergence order p , the asymptotic error is expected to behave according to 75:

$$E_h \approx Ch^p \quad (75)$$

where C is approximately independent of $h \sim \frac{1}{N-1}$, where N is the number of intervals per coordinate direction on the unit square and p is the convergence rate [19].

Taking logarithms gives:

$$\ln E_h \approx \ln C + p \ln h \quad (76)$$

which means that a plot of $\ln E_h$ against $\ln h$ will approach a straight line whose slope is p [37].

Hence the general formula for the convergence rate will take the form of 77.

$$p = \frac{\ln\left(\frac{E_1}{E_2}\right)}{\ln\left(\frac{h_1}{h_2}\right)} \quad (77)$$

When the mesh size is being halved $h_2 = \frac{h_1}{2}$ the local order of convergence p will be derived like it is shown in 78.

$$\begin{aligned} E_1 &\approx Ch_1^p \quad \text{and} \quad E_2 \approx C\left(\frac{h_1}{2}\right)^p \\ &\implies \frac{E_1}{E_2} = 2^p \\ \therefore p &= \log_2\left(\frac{E_1}{E_2}\right) \end{aligned} \quad (78)$$

For sufficiently fine meshes, the observed convergence rate is expected to approach the theoretical values 2 and 4 for P1 and P2 elements, respectively [19].

This method is used because it does not use complex calculations or depend on the factors we cannot control, it is based strictly on the computed values and knowledge of the theory.

The results show that the coarse meshes produce relatively large eigenvalue errors. As the mesh is refined, the computed eigenvalues approach their analytical values. This behaviour agrees with the finite element convergence theory developed earlier.

The log-log convergence plot provides a visual confirmation of the theoretical convergence rates [37]. For the P1 approximation, the calculated convergence rates approach 2, which is consistent with the theoretical eigenvalue error estimate $O(h^2)$. For the P2 approximation, the convergence rates approach 4, confirming the theoretically predicted $O(h^4)$ behaviour.

Although P2 elements require more degrees of freedom per element, their higher convergence rate allows the same accuracy to be achieved with a considerably coarser mesh than for P1 elements.

A. Laplace results

For the Laplace eigenproblem we obtained results for P1 and P2 elements. The results are presented in Table I and in Table II.

TABLE I: P1 Error dependency on the number of points per axis.

N points per axis	Interior DoF	Computed λ_1	$ \lambda_1 - \lambda_{1,h} $
3	1	48.000000	28.260791
6	16	21.640205	1.900996
12	100	20.127062	0.387853
24	484	19.826257	0.087048
48	2116	19.759917	0.020708

TABLE II: P2 Error dependency on the number of points per axis.

N points per axis	Interior DoF	Computed λ_1	$ \lambda_1 - \lambda_{1,h} $
3	9	19.94133	0.202126
6	81	19.76407	0.024864
12	441	19.74036	0.001153
24	2025	19.73927	0.000062
48	8649	19.73921	0.000004

From the tables we can see that with mesh refinement the error decreases for both P1 and P2 elements. However, we can spot that the absolute error of P1 elements with 48 points per axis is nearly the same as the error of P2 elements with 6 points per axis.

Additionally, we can compute the relative error, using 79.

$$e_{rel} = \frac{|\lambda - \lambda_h|}{\lambda} \quad (79)$$

At $N = 48$ the P1 result has $e_{rel} \approx 1.05 \times 10^{-3}$ which is about 0.105%, while the P2 result gives us $e_{rel} \approx 2.03 \times 10^{-7}$ which is about 0.00002%. This shows how much more precise P2 elements are.

The results empirically demonstrate the higher accuracy obtained with P2 elements for comparable mesh resolutions. This illustrates how the higher polynomial order can achieve a given accuracy with substantially fewer elements, although P2 elements can become more "expensive" computationally since they introduce additional degrees of freedom.

B. Convergence

As it is presented in Fig. 9 and Fig. 10 the convergence plots approach straight lines on the log-log scale. The observed slopes also become closer to the theoretical convergence rates as the mesh is refined. The coarser meshes may exhibit some derivation from asymptotic behaviour because the discretisation error is not yet sufficiently small.

The plots approach straight-line behaviour on the log-log scale as the mesh is refined. The corresponding slopes approach the theoretical values of 2 for P1 and 4 for P2.

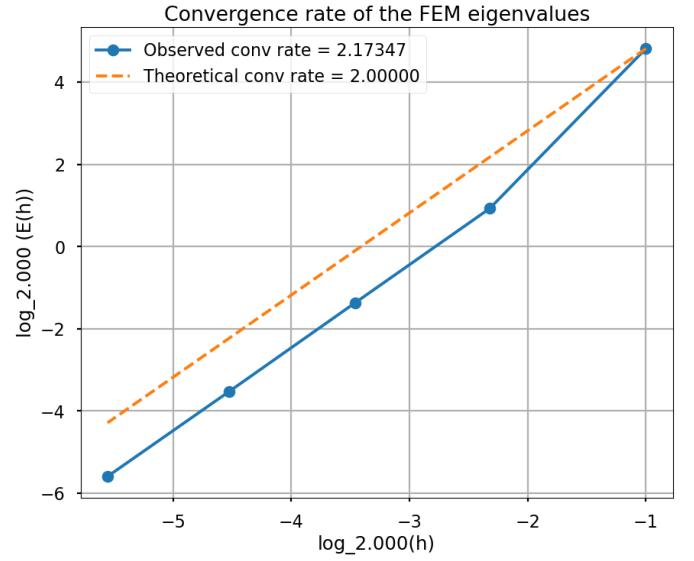


Fig. 9: P1 element approximation convergence

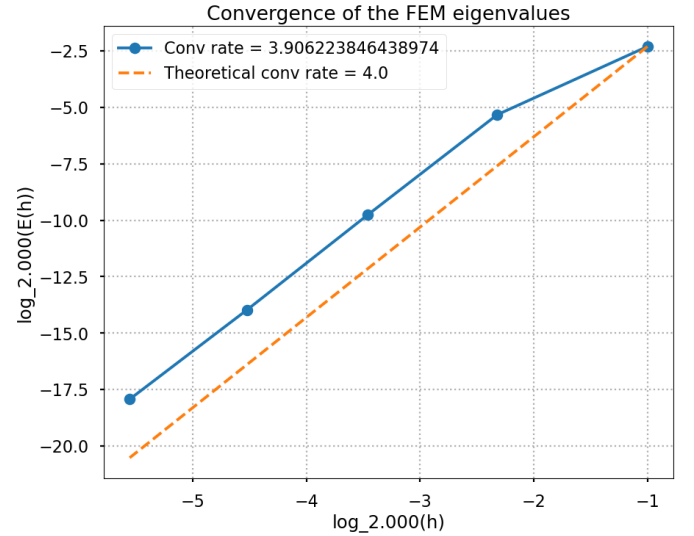


Fig. 10: P2 element approximation convergence

VIII. LAPLACE RESULTS: EIGENVECTORS

The eigenvalues alone do not provide the complete solution of the eigenproblem. Each eigenvalue is associated with one or more eigenfunctions and therefore the accuracy of the computed eigenfunctions must also be checked.

For a non-degenerate eigenvalue, the corresponding eigenspace is one-dimensional. Hence, after consistent normalisation and sign alignment, the numerical eigenfunction can be compared directly with the particular analytical eigenfunction. Since eigenfunctions are defined only up to multiplication by a non-zero constant, the numerical eigenfunction must first be normalised consistently with the analytical solution.

For the unit square with homogeneous Dirichlet boundary

condition, the analytical eigenfunctions have the form of 80

$$u_{m,n}(x,y) = C \sin(m\pi x) \sin(n\pi y), \quad m,n = 1, 2, \dots \quad (80)$$

where C is a normalisation constant.

When talking about eigenfunctions we want to see its shape and understand what it means. So in the figures below the shape of eigenfunction with 24 points per axis are presented in the figures below in 2D Fig. 11 and 3D Fig. 12. The computed eigenfunctions are visualised using two-dimensional and three-dimensional representations. The P1 approximation is shown using a two-dimensional surface/colour representation, while the P2 approximation is shown as a three-dimensional surface to make its higher-order variation more apparent.

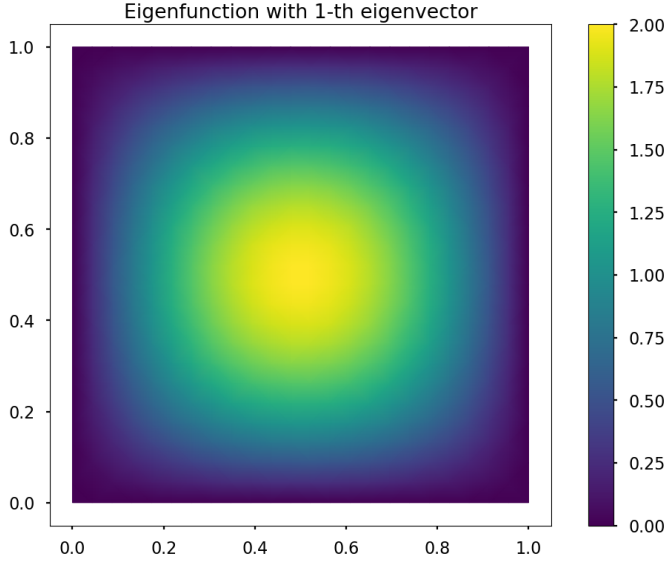


Fig. 11: First eigenfunction in 2D built with P1 elements

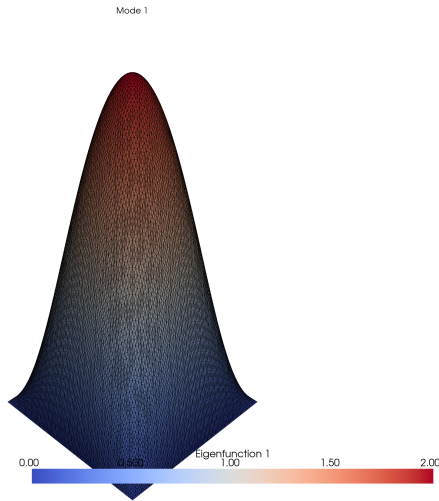


Fig. 12: First eigenfunction in 3D built with P2 elements

The first eigenfunction has no interior nodal lines and contains a single positive maximum. This is characteristic of

the fundamental mode of the Dirichlet Laplacian. In applications where the Laplacian represents a vibration problem, this corresponds to the fundamental mode of vibration.

Another question that remains is how precise our obtained graphs? We will use the L^2 - 82 and H^1 -norms 83 of the error to quantify the difference between the numerical and analytical eigenfunctions. Very important note for the computations would be the sign check, because both positive and negative eigenvectors are meaningful, they just point in opposite direction we need to make sure that the negative eigenvector will not make the error massive. To ensure consistent orientation, we evaluate the discrete L^2 inner product:

$$(u_h, u)_M = \vec{u}_h^T M \vec{u} \quad (81)$$

If this quantity is negative, the sign of the numerical eigenfunction is reversed by replacing u_h with $-u_h$. We are using mass matrix M because we are approximating an L^2 inner product, not an ordinary Euclidean dot product.

If the dot product is negative that means that the approximation u_h points in different direction from the exact u . Physically and mathematically that is absolutely normal, however, it will make the error enormous. Hence, we prevent this by multiplying our u_h by -1 without changing the corresponding eigenvalue or eigenspace.

In addition to this, it should be mentioned that all the vectors should be normalised to prevent wrong computed errors.

$$\|u - u_h\|_{L^2(\Omega)} = \sqrt{\int_{\Omega} |u - u_h|^2 dx} \quad (82)$$

$$\begin{aligned} \|u - u_h\|_{H^1(\Omega)} &= \sqrt{\|u - u_h\|_{L^2(\Omega)}^2 + \|\nabla(u - u_h)\|_{L^2(\Omega)}^2} \\ &= \sqrt{\int_{\Omega} |u - u_h|^2 dx + \int_{\Omega} |\nabla u - \nabla u_h|^2 dx} \end{aligned} \quad (83)$$

Graphically, even on the coarse mesh (6 points per axis) our approximation with both P1 Fig. 13 and P2 elements Fig. 14 look very similar to the exact.

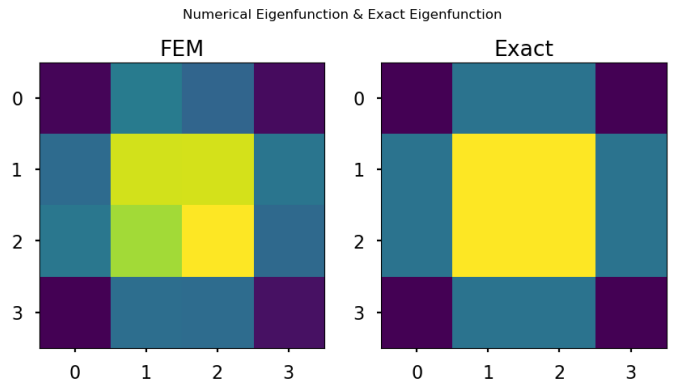


Fig. 13: Numerical and exact first eigenfunctions obtained using P1 elements

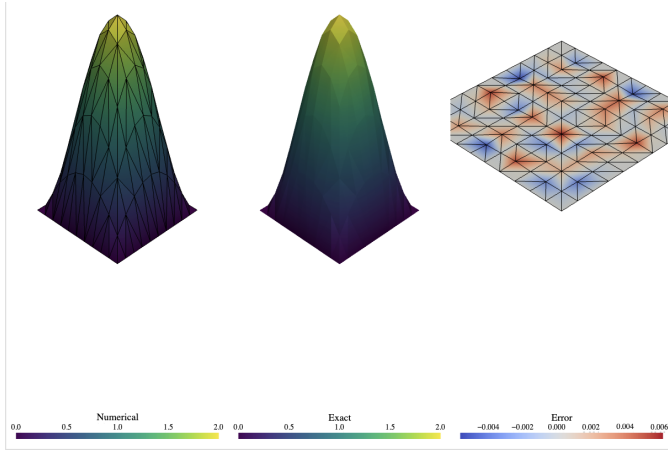


Fig. 14: Numerical and exact first eigenfunctions obtained using P2 elements

For P1 elements we can see slight difference in the shade of yellow in the middle which assures us that even on the coarse mesh we are getting quite good approximation.

The P2 approximation is visually closer to the analytical solution, which is consistent with the smaller numerical errors observed in the following error analysis.

Of course graphical analysis is important, but not yet sufficient, we need to look at the errors of both approximations and their convergence to be sure that we meet the theoretical data.

From the code we can create Table III for P1 element approximation and Table IV for P2.

TABLE III: L^2 and H^1 errors for P1 element approximation

N points per axis	L^2 error	H^1 error
3	0.000000	0.000000
6	0.020375	0.407897
12	0.003773	0.150931
24	0.000886	0.073849
48	0.000206	0.037457

TABLE IV: L^2 and H^1 errors for P2 element approximation

N points per axis	L^2 error	H^1 error
3	0.011323	9.150693
6	0.001789	0.159279
12	0.000153	0.034025
24	0.000016	0.007874
48	0.000002	0.001884

For the eigenvalue error, the P2 approximation with only 6 points per axis achieves an error comparable to the P1 approximation with 48 points per axis. For the L^2 eigenfunction error, however, a P2 mesh with 12 points per axis is required to obtain an error comparable to the P1 result with 48 points per axis.

The errors with 3 points per axis should not be taken in consideration, since we do not have any degrees of freedom, we

accidentally pointed in the right place, which is not a proof of good approximation. For this reason, the reported comparison does not provide a meaningful convergence measurement.

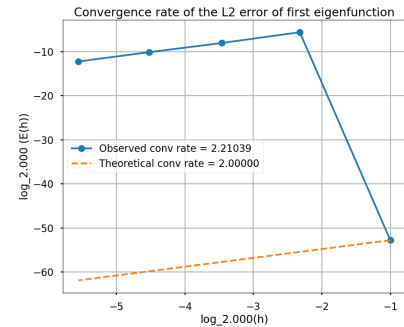
Very reasonable question may arise - why do we need H^1 error if it is much larger than L^2 , what does it show to us?

The L^2 measures the height (value) of a function, whereas the H^1 norm additionally measures the error in its first derivatives.

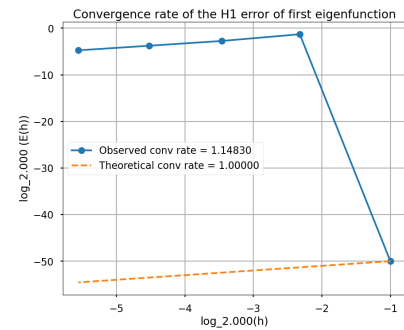
With eigenfunctions low L^2 error is not enough because, firstly, a graph can stay close to correct values while bouncing up and down, this will create a jagged, rough line with high H^1 error, while L^2 error will stay low. Additionally, if the derivative error is high our approximation will give us wrong predictions, in other words, it will predict the right average position, but the wrong speed, pressure or stress. For PDEs in which derivatives of the solution have direct physical significance, such as gradients, fluxes or strains, controlling H^1 error is important.

The last thing to check is the convergence rate: for the L^2 the convergence rate is $O(h^{p+1})$, so the slope of the line for P1 approximation should be around 2 and 3 for P2; for the H^1 the convergence rate is $O(h^p)$, so the slope of the line for P1 approximation should be around 1 and 2 for P2.

As we can see from Fig. 15 both L^2 15a and H^1 15b are approaching theoretical convergence rate. The jump on the right side happens because of first approximations which have lower accuracy and higher convergence rate.



(a)



(b)

Fig. 15: First P1 approximated eigenfunction (a) L^2 , (b) H^1 error convergence

Same result we are seeing for P2 elements on the Fig. 16, which provides numerical evidence that the implementation is consistent with the theoretical convergence behaviour.

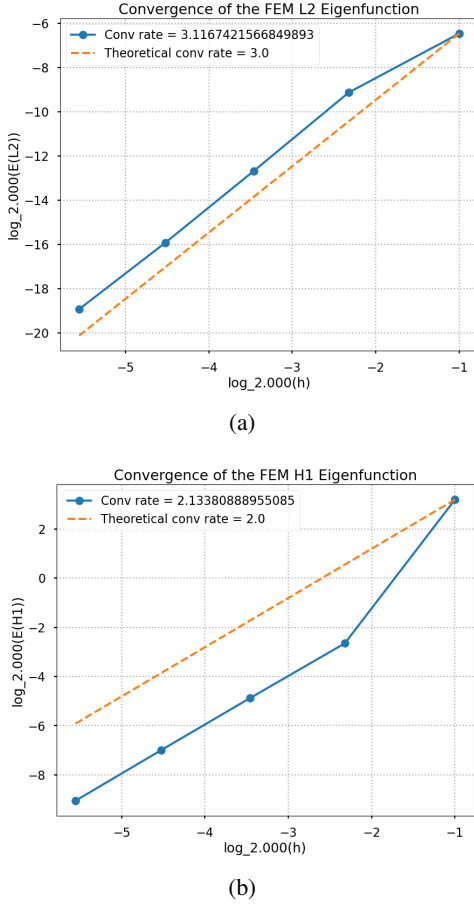


Fig. 16: First P2 approximated eigenfunction (a) L^2 , (b) H^1 error convergence

A. Degenerate eigenspaces

The comparison becomes more complicated for degenerate eigenvalue. In this case, an eigenvalue is associated with more than one linearly independent eigenfunction. Therefore, the individual numerical eigenfunctions cannot necessarily be expected to coincide with the particular analytical eigenfunctions chosen for comparison.

For the unit square, the second and third analytical eigenvalues are identical:

$$\lambda_{1,2} = \lambda_{2,1} = 5\pi^2 \quad (84)$$

The corresponding eigenspace is therefore two-dimensional and is spanned by the two linearly independent eigenfunctions 85 and 86.

$$u_1(x, y) = \sin(\pi x) \sin(2\pi y) \quad (85)$$

$$u_2(x, y) = \sin(2\pi x) \sin(\pi y) \quad (86)$$

Same as for the non-degenerate eigenfunction we first want to have an idea of the shape of these functions. In Fig. 17 we

can see the 2D plot of the two eigenfunctions with 24 points per axis.

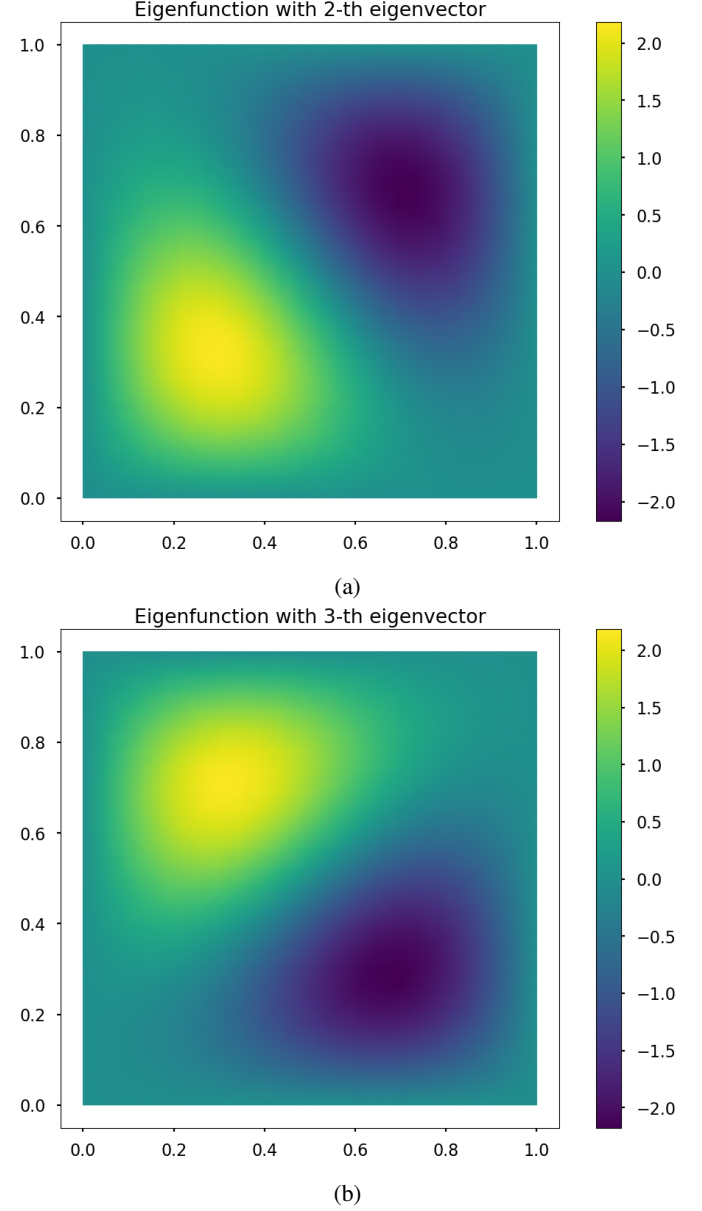


Fig. 17: Degenerate eigenfunctions approximated with P1 elements (a) second eigenfunction, (b) third eigenfunction

And the 3D P2 approximation is presented in Fig. 18.

As we can tell the third eigenfunctions on Fig. 17b and Fig. 18b are rotated comparing to the second ones. This happens because the second eigenvalue is repeated, so the numerical eigensolver does not return $u_{2,1}$ or $u_{1,2}$, it returns some arbitrary rotation of the 2D eigenspace basis.

In order to spot this rotation for P1 elements the program returned the Table V, where we can analyse the dot products between numerical and exact eigenfunctions.

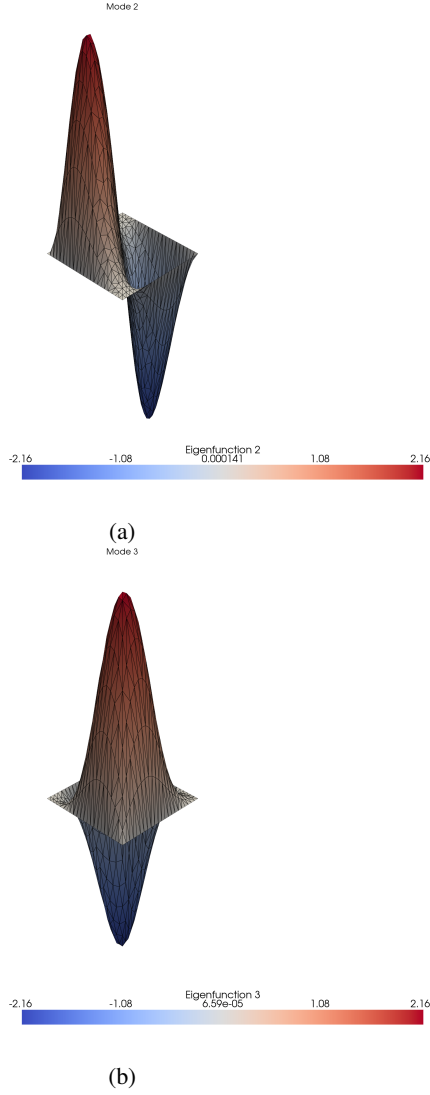


Fig. 18: Degenerate eigenfunctions approximated with P2 elements (a) second eigenfunction, (b) third eigenfunction

TABLE V: Dot products between exact and numerical eigenfunctions

N	$u_{1,2} \dot{u}_{h,1,2}$	$u_{2,1} \dot{u}_{h,2,1}$	$u_{1,2} \dot{u}_{h,2,1}$	$u_{2,1} \dot{u}_{h,1,2}$
1	0.609668	0.590639	0.803962	0.790167
2	0.595032	0.593817	0.804533	0.803643
3	0.610267	0.610161	0.792274	0.792193
4	0.842743	0.842736	0.538327	0.538316

The observed values indicate that the numerical eigenvectors are rotated combinations of the exact eigenfunctions, which is the expected behaviour for a degenerate eigenspace. Additionally, dot products are not close to 1, because the numerical eigenvectors are a rotation of the exact basis by some angle θ . Dot product gives the $\cos(\theta)$ and $\sin(\theta)$. If we look at the table we will see that the $(u_{1,2} \dot{u}_{h,1,2})^2 + (u_{1,2} \dot{u}_{h,2,1})^2 \approx 0.6^2 + 0.8^2 \approx 1$, which is Pythagoras theorem. Hence,

although neither numerical eigenvector individually coincides with particular analytical eigenfunction, together they provide evidence that the correct eigenspace has been captured..

The jump up to 0.84 in the last row can be explained that the eigensolver is free to choose any orthonormal basis of the approximate 2D eigenspace. However, small changes in the discretisation may therefore lead to different rotations of the numerical basis, even though the eigenspace itself is converging correctly. That is why convergence of individual eigenvectors is not an appropriate measure for repeated eigenvalue. Instead the convergence should be assessed at the level of the eigenspace itself. And since the individual eigenfunctions associated with degenerate eigenvalue are not uniquely determined, comparing the numerical eigenfunctions with one particular analytical eigenfunction is not an appropriate measure of the eigenspace error.

Since the numerical eigensolver may return any orthonormal basis of the degenerate eigenspace, the individual numerical eigenfunctions should not be compared with the particular analytic eigenfunctions. Instead, we should consider measuring the distance between numerical eigenfunction u_h and the exact eigenspace.

This distance is obtained by projecting u_h onto E 87.

$$E = \text{span}\{u_1, u_2\} \quad (87)$$

Known that the analytical eigenfunctions u_1 and u_2 are orthogonal in $L^2(\Omega)$ the projection of u_h onto E is given by 88.

$$P_E u_h = \sum_{i=1}^2 \frac{(u_h, u_i)_{L^2}}{(u_i, u_i)_{L^2}} u_i \quad (88)$$

So the error will represent the distance between the exact eigenspace and our projection, which can be measured by 89.

$$d(u_h, E) = \|u_h - P_E u_h\|_{L^2(\Omega)} \quad (89)$$

This value approaches zero with the finer mesh, which indicates that the numerical eigenfunction approaches the exact eigenspace, independently of the particular basis chosen by the eigensolver.

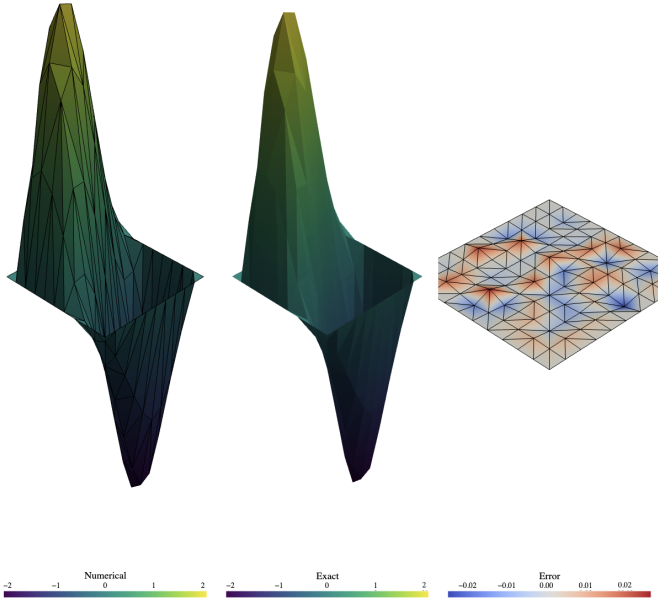
The same approach can be extended to the H^1 -norm, allowing the distance between the numerical eigenfunction and the exact eigenspace to be assessed in both the L^2 and H^1 settings.

Before we proceed to the computed errors, firstly, we need to get the visual perspective of the difference between numerical and exact eigenfunctions which is shown in Fig. 19.

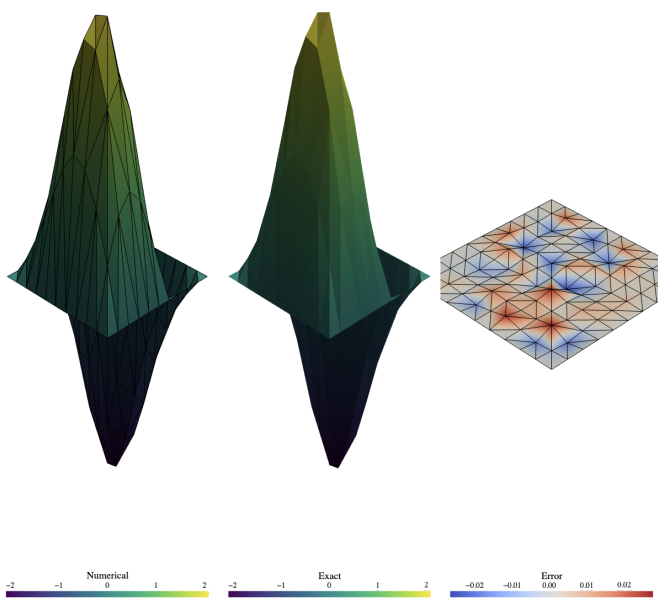
As we can see on the relatively fine mesh (6 points per axis) we are getting quite accurate visual approximation.

But what about computed errors?

In the Table VI and Table VII we are given the L^2 -norm of the projection distance of the P1 and P2 approximations.



(a) Second eigenfunction



(b) Third eigenfunction

Fig. 19: Degenerate eigenfunctions approximated with P2 elements against exact

TABLE VI: L^2 -norm of the projection distance of the P1 degenerate eigenfunction

N points per axis	$\ P_e v_{1,2} - u_{1,2}\ _{L^2}$	$\ P_e v_{2,1} - u_{2,1}\ _{L^2}$
3	NaN	NaN
6	0.066759	0.073152
12	0.010034	0.010344
24	0.002278	0.002248
48	0.000522	0.000516

As we can see, errors in both second and third eigenfunc-

tions decrease similarly and have very close numbers, so for P2 approximation it was decided to take the biggest value of both and compare with exact, because it is better to have some margin of error.

TABLE VII: L^2 -norm of the projection distance of the P2 degenerate eigenfunction

N points per axis	$\max\{\ P_e v_{1,2} - u_{1,2}\ _{L^2}, \ P_e v_{2,1} - u_{2,1}\ _{L^2}\}$
3	NaN
6	0.007687
12	0.000602
24	0.000063
48	0.000007

Interestingly, we can spot the same idea as in the non-degenerate case, the error of the P1 elements with 48 points per axis is equivalent to the error of P2 elements with 12 points per axis. So, degeneracy does not have an influence on the error.

Similarly we obtained the errors for H^1 -norm projection error for the P1 and P2 approximations.

For the P1 element to find the H^1 error the small trick was used. For any nodal vector w representing a piecewise-linear function, the discrete H^1 norm can be found using 90.

$$\|w\|_{H^1} = w^T M w + w^T A w = w^T (M + A) w \quad (90)$$

Where M is a mass matrix and A is a stiffness matrix or the discrete bilinear form $\int \nabla u \nabla v$. This approach allows us to avoid finding centroid and do complex computations with quadrature. The P1 matrix trick, only ever sees nodal values, so it exactly computes the H^1 norm of the difference between the piecewise-linear interpolant of u and u_h , not u itself. But the main question, will the different computation influence the result?

No, Because by the triangle inequality $\|u - u_h\|_{H^1} \leq \|u - I_h u\|_{H^1} + \|I_h u - u_h\|_{H^1}$, and interpolation error $\|u - I_h u\|_{H^1}$ for P1 is itself $O(h)$ — same order as the FEM discretisation error we are trying to measure. So the two quantities are asymptotically equivalent in convergence rate (both $O(h)$), even though their absolute numerical values at a given number of points per axis will not match exactly.

TABLE VIII: H^1 -norm of the projection distance of the P1 degenerate eigenfunction

N points per axis	$\ P_e v_{1,2} - u_{1,2}\ _{H^1}$	$\ P_e v_{2,1} - u_{2,1}\ _{H^1}$
3	NaN	NaN
6	1.146420	1.142827
12	0.420973	0.394877
24	0.186250	0.189181
48	0.093434	0.094670

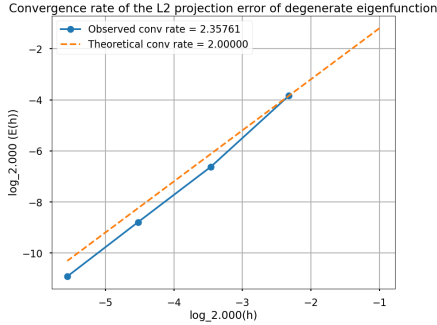
TABLE IX: H^1 -norm of the projection distance of the P2 degenerate eigenfunction

N points per axis	$\max\{\ P_e v_{1,2} - u_{1,2}\ _{H^1}, \ P_e v_{2,1} - u_{2,1}\ _{H^1}\}$
3	NaN
6	0.567328
12	0.123601
24	0.028430
48	0.006829

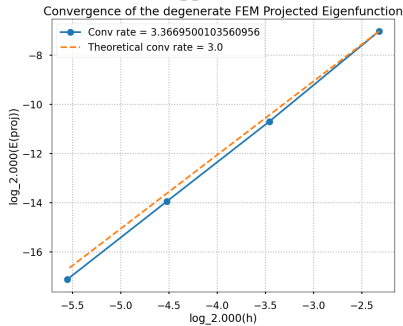
In comparison, the H^1 -norm error requires more points per axis in P2 approximation in order to get the same precision as in P1 elements. For the coarser mesh, the difference between the two approximations is relatively small, while with further refinement the higher-order P2 approximation exhibits a substantially smaller H^1 error. This is consistent with the higher-order convergence of P2 elements.

The last check that the project requires is the convergence check, the L^2 projection error norm convergence rate is $O(h^{p+1})$, so for P1 elements we are expecting to see the line on the log-log plot with a slope to be around 2 and for P2 elements slope should be approximately 3. The H^1 projection error norm convergence rate is $O(h^p)$, so we are expecting to see the slope of the line 1 for P1 elements and 2 for P2.

As we saw from the errors of P1 elements, their numbers are very close, so for the convergence we can take the maximum out of two and plot it. In Fig. 20 and Fig. 21 we can see that the observed convergence rates are consistent with the theoretical rates providing strong numerical evidence that the implementation is behaving as expected.



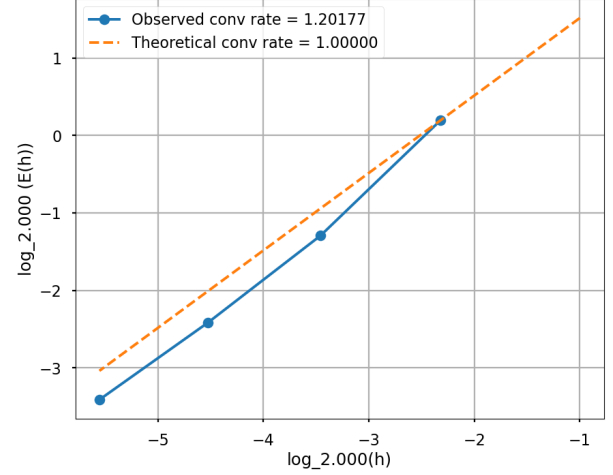
(a) P1 approximation



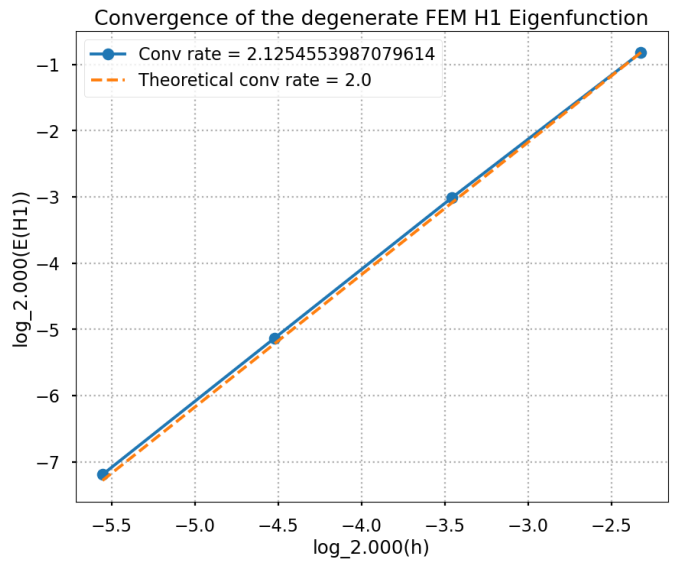
(b) P2 approximation

Fig. 20: L^2 -norm convergence

Convergence rate of the H1 projectionerror of degenerate eigenfunction



(a) P1 approximation



(b) P2 approximation

Fig. 21: H^1 -norm convergence

IX. LAPLACE EIGENPROBLEM CONCLUSION

From the results presented it is clear that P2 elements approximation converges faster, but it has some drawbacks. The main one is complexity in implementation: to write code for the P1 elements approximation is much easier than for P2, due to the need of using the gradients and quadratures for approximating integrals. Moreover, P2 elements have more degrees of freedom, which requires more memory and time to proceed with computations. As it was mentioned before the errors for both eigenvalues and eigenvectors of P1 approximation with 48 points per axis is equivalent to P2 approximation with 12 points per axis. So each program was executed 5 times and the average time was taken, the P1 approximation with 48 points per axis averaged at 3.6 seconds, while P2 approximation with 12 points per axis averaged at 4.7 seconds.

At the same time if we do the same test for P2 approximation with 48 points per axis we will average at about 65 seconds.

From this we can conclude that P1 elements are easier to implement, they produce fast and quite accurate results and they are easy to explain, at the same time P2 elements producing very accurate approximations and allow to have more detailed picture about the unknown function.

The results therefore illustrate a trade-off between approximation order and computational cost. P2 elements achieve significantly smaller errors on relatively coarse meshes, but each element introduces additional degrees of freedom and requires a more involved assembly procedure. Consequently, although P2 can achieve a prescribed accuracy with fewer mesh refinements, this does not necessarily imply a lower total computational cost for every problem.

X. NUMERICAL RESULTS FOR THE SCHRÖDINGER'S EIGENPROBLEM

As it was mentioned before in the finite element formulation used in this project, Schrödinger eigenproblem has the same stiffness and mass matrices as the Laplace eigenproblem, with an additional potential matrix arising from the potential energy term. However, potential energy has one very important detail - it can be defined using different equations, which will change the physical meaning and will require some adjustments in the computer implementation. The main questions are how does each variant of the potential influence the results and what do these results mean? In this project 3 potential models will be considered: infinite square well, quantum harmonic oscillator and 2D Coulomb radial equation. Very important note is that the computational experiments use a scaled/non-dimensional Schrödinger operator, in order to analyse FEM as mathematical tool for solving partial differential equations.

A. Infinite square well

Infinite square well is the model where the particle is confined in a single region. The potential is constant inside the well and infinite outside. The zero of potential energy can be chosen arbitrary, so it is convenient to set the potential inside the well to zero. The easiest to visualise would be 1D infinite square well illustration which is shown in Fig. 22, which restricts particle to move only in x direction. Hence, in our 2D case, the particle will be restricted to move around the $x - y$ plane.

Moreover, the infinite potential outside the well does not need to be represented explicitly. Instead, confinement is imposed through homogeneous Dirichlet boundary condition which ensures that we will not have a solution on the boundary.

This case is considered the simplest of the Schrödinger eigenproblem because the only change that happens is the addition of potential matrix to the stiffness matrix from 91 where every element of the matrix is a constant.

$$(A + K_V)\psi = \lambda M\psi \quad (91)$$

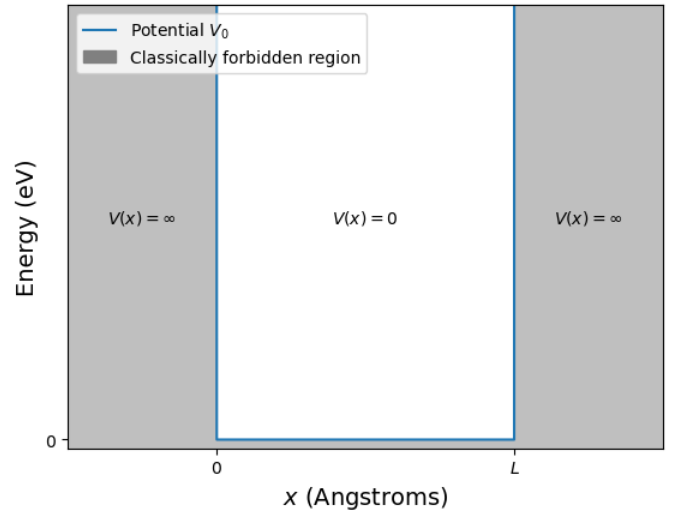


Fig. 22: 1D infinite square well [38]

Where $K_V = \int_{\Omega} V \phi_i \phi_j dx = V M_{i,j}$. This allows us to rewrite 91 as:

$$(A + VM)u = (\lambda_{Lap} + V)Mu \quad (92)$$

Therefore $\lambda_{Sch} = \lambda_{Lap} + V$

and the eigenvectors are unchanged.

To compare results from Laplace eigenproblem with Schrödinger eigenproblem with constant potential, we made potential equal to 5. Since we know that the Laplace implementation converges to the theoretical values we will not check the convergence for Schrödinger problem with constant potential. The comparison is presented in Table X.

TABLE X: Laplace eigenvalues and Schrödinger eigenvalues with $V = 5$

$\lambda_{1,Lap}$	$\lambda_{1,Sch}$	$ \lambda_{1,Lap} - \lambda_{1,Sch} $	$\lambda_{2,3Lap}$	$\lambda_{2,3Sch}$	$ \lambda_{2,3,Lap} - \lambda_{2,3,Sch} $
19.94133	24.94133	5.00000	57.311070	62.311070	
19.76407	24.76407	5.00000	49.682181	54.682181	
19.74036	24.74036	5.00000	49.363364	54.363364	
19.73927	24.73927	5.00000	49.348834	54.348834	
19.73921	24.73921	5.00000	49.348069	54.348069	

As we can see all the eigenvalues shifted by the amount of the potential. But what about eigenfunctions? We will check the eigenfunctions on the coarsest mesh, simply because finer meshes will produce more dimensional vectors and it would be difficult to write them down to analyse and moreover all we need is just to see the shift or trend in changes.

The two matrices in 93 show the column spaces of first

three eigenfunctions.

$$\begin{bmatrix} -2.03449568 \\ -0.981551976 \\ -1.43986680 \\ -0.981551976 \\ -1.43986680 \\ -0.981551976 \\ -1.43986680 \\ -1.43986680 \\ -0.981551976 \end{bmatrix} \begin{bmatrix} -2.03449568 \\ -0.981551976 \\ -1.43986680 \\ -0.981551976 \\ -1.43986680 \\ -0.981551976 \\ -1.43986680 \\ -1.43986680 \\ -0.981551976 \end{bmatrix} \quad (93)$$

The first eigenfunctions are identical, which means that potential matrix does not affect the eigenspaces corresponding to the eigenvalues, in other words, with constant potential, the potential matrix is proportional to the mass matrix, that is why the eigenfunctions remain unchanged while all eigenvalues are shifted by the same amount. Unfortunately, We can not analyse the degenerate pair, because on this coarse mesh (3 points per axis) there is no degeneracy, and the next mesh produces much larger eigenfunctions which will become a challenge to put in this report.

B. Quantum harmonic oscillator

The quantum harmonic oscillator models a particle in a parabolic potential well, featuring discrete, equally spaced energy levels and a non-zero ground state energy [39].

The main difference between infinite square well and quantum harmonic oscillator is that in harmonic oscillator we have quantised energy levels that are given by 94.

$$E_{n_x, n_y} = \hbar\omega(n_x + n_y + 1) \quad \text{where } n_x, n_y = 0, 1, 2, \dots \quad (94)$$

Hence, because our problem is not on the infinite line, so our potential energy will be defined as $V(x, y) = \frac{1}{2}m\omega^2((x - x_0)^2 + (y - y_0)^2)$, where ω is angular frequency and m is the mass of the particle [39].

There are few changes that differ this problem from all previous problems. First of all, as it was mentioned before, because we define $V(x, y)$ with equation that contains unknown variables: x and y , and the basis functions $\phi_i(x, y)$ and $\phi_j(x, y)$ also depend on x and y , multiplying all of them, we can get polynomial up to degree 6. The previous quadrature rule is exact only for polynomials up to degree 4 and is therefore insufficient to integrate this degree-6 integrand exactly. Additionally, there is no simple analytical solution for the harmonic oscillator on the finite square domain. Hence, to check whether the FEM eigenvalues are correct, we need a "numerical reference", in other words we need to solve the same problem with trusted numerical method and compare with FEM implementation. Luckily, this 2D problem can be solved easily with few numerical methods.

To start, we reorganise our potential energy formula like it is done in 95 (we will use the scaled version, so all physical constants, such as m are absent).

$$\begin{aligned} V(x, y) &= \frac{1}{2}\omega^2((x - x_0)^2 + (y - y_0)^2) \\ &= \frac{1}{2}\omega^2(x - x_0)^2 + \frac{1}{2}\omega^2(y - y_0)^2 \end{aligned} \quad (95)$$

So, now we can separate 2D problem into two identical 1D problems 96 and solve them on $0 < x < 1$ and $0 < y < 1$ and discretised domain with arbitrary number of interior grid points.

$$-u''(x) + \frac{1}{2}\omega^2(x - x_0)^2 u(x) = Eu(x) \quad (96)$$

We know that the second derivative can be approximated with finite differences 97.

$$-u''(x_i) \approx \frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} \quad (97)$$

After we solve each 1D problem individually using *scipy.linalg.eightridiagonal*. We get eigenvalues and eigenfunctions. With eigenvalues in order to go from 1D to 2D we can simply add them together, which mathematically looks like this:

$$\begin{aligned} H_x \phi_{n_x} &= \lambda_m^{(x)} \phi_{n_x}, \quad H_y \phi_{n_y} = \lambda_n^{(y)} \phi_{n_y} \\ \therefore \lambda_{n_x, n_y} &= \lambda_{n_x}^{(x)} + \lambda_{n_y}^{(y)} \end{aligned} \quad (98)$$

And 2D eigenfunctions are computed differently - for a separable problem, the 2D eigenfunction has the form:

$$\psi_{m,n}(x, y) = \phi_m(x) \phi_n(y) \quad (99)$$

However, there is one issue, the finite difference method uses more grid points than the FEM implementation, which will cause an error, if implemented without adjustments. We need to interpolate the 1D reference function, we take the point x_i from FEM and check the value of 1D reference function at this x_i , which mathematically looks like this:

$$\psi_{m,n}^{ref}(x_i, y_i) = \phi_m^{ref}(x_i) \phi_n^{ref}(y_i) \quad (100)$$

Another important note is normalisation. The *eightridiagonal* returns the Euclidean norm, while we need the L^2 norm 81, so normalisation has the form:

$$u_{ref} = \frac{u_{ref}}{\sqrt{u_{ref}^T M u_{ref}}} \quad (101)$$

1) *Results: eigenvalues:* In the implementation we took $\omega = 160$, which gives a sufficiently confining potential that produces oscillator-like low-energy states while remaining convenient for numerical computations. $(x_0, y_0) = (0.5, 0.5)$ to have the minimum potential in the middle of the unit square. The 1D reference problem is solved using a sufficiently fine finite-difference grid to obtain eigenvalues accurate to approximately four decimal places.

In the Table XI eigenvalues and their errors are shown.

TABLE XI: Quantum harmonic oscillator eigenvalues

λ_1	$ \lambda_1 - \lambda_{1,ref} $	$\lambda_{2,3}$	$ \lambda_{2,3} - \lambda_{2,3,ref} $
279.91036	53.636212	919.984171	467.435906
237.45286	11.178717	512.878866	60.330601
227.15517	0.881020	456.099324	3.551059
226.32583	0.051687	452.754458	0.206193
226.27727	0.003124	452.560660	0.012395

Here, λ_1 is the lowest eigenvalue of the scaled finite-domain quantum harmonic oscillator problem, while $\lambda_{2,3}$ denotes the degenerate first excited level.

2) *Results: eigenfunctions:* The FEM eigenvector $c = (c_1, \dots, c_N)^T$ contains the coefficients to construct the approximate wavefunction ψ as in 102, whose square $|\psi|^2$ shows the probability density of finding the particle.

$$\psi_h(x, y) = \sum_{i=1}^N c_i \phi_i(x, y) \quad (102)$$

In Fig. 23 we can see the plots of wavefunctions at the ground state (left) and two higher energy level states (middle and right).

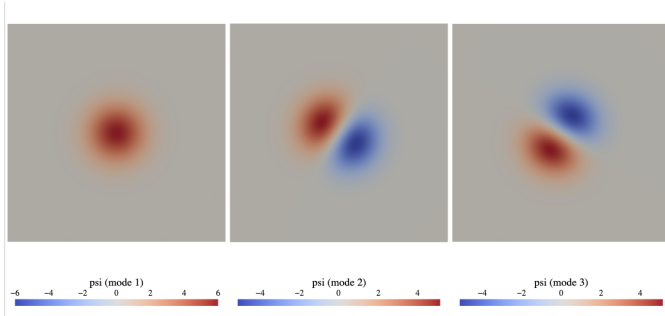


Fig. 23: Wavefunctions at different quantum states

The wavefunctions ψ are the probability amplitudes, but what does the sign of the wavefunction tell us? Whenever ψ crosses zero (the red/blue boundary) is a node. Node count is important because it allows us to read the excitation level: mode 1 has zero interior sign changes, hence it is a ground state. Modes 2 and 3 each have exactly one nodal line, therefore they are the first excited level. This also meets perfectly with theory, because for a 2D isotropic harmonic oscillator, every level n is $(n+1)$ — so $n=0$ gives 1 state, $n=1$ gives 2, that is what we see in the figure.

Another useful characteristic of the wavefunction is its square $|\psi|^2$ that represents the probability density, which informally means where are we most likely to find the particle. The actual probability P can be calculated using 103.

$$P = \int_{\Omega} |\psi(x, y)|^2 dx dy \quad (103)$$

It is clear since probability density is the square of the wavefunction, the plot will look similar, but without negative signs. However, if we use only $|\psi|^2$ plots for interpreting results we can miss on the nodal count. Because we will not be able to say if the dark seam is the node or it is just a

separation of two low-density blobs. To be exact we can look at Fig. 24 and notice that we cannot tell if it is a continuous function crossing zero, with the two sides carrying opposite phase or not.

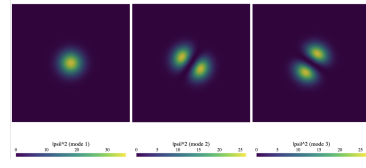
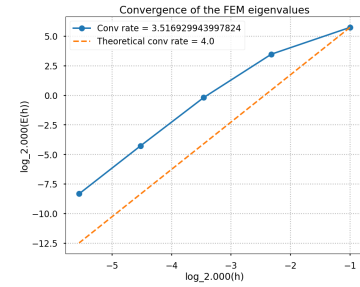


Fig. 24: Probability density at different quantum states

3) *Convergence:* With constant potential it was clear that convergence will go similarly as the Laplace eigenproblem, however, will additional parameters influence the convergence?

As it was expected, both eigenvalues and eigenvectors errors converged at the theoretical rates, which we can see in Fig. 25.



(a) Eigenvalue

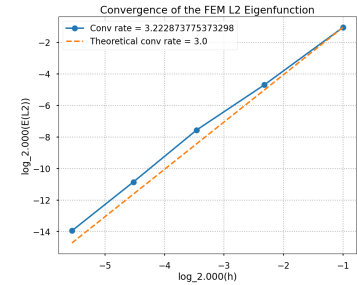
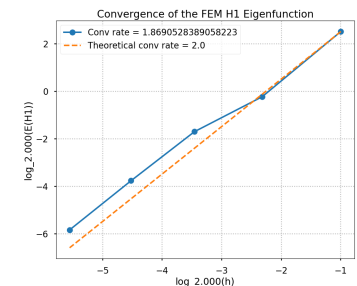

 (b) Eigenfunction L^2 -norm projection

 (c) Eigenfunction H^1 -norm projection

Fig. 25: Quantum harmonic oscillator convergence rates

C. Coulomb potential and radial formulation

Coulomb potential is defined as:

$$V(r) = -\frac{Z}{r}, \quad r = \sqrt{(x - x_0)^2 + (y - y_0)^2} \quad (104)$$

Where Z is the atomic number, that is equal to amount of protons in the nucleus [40]. For the physical hydrogen atom, the Coulomb potential energy term is proportional to $-\frac{Ze^2}{4\pi\epsilon}$ where e is the elementary charge and ϵ is vacuum permittivity [41]. In this project we use scaled mathematical model in which the relevant physical constants are absorbed into scaling, giving $V(r) = -\frac{Z}{r}$. In addition to this, since the domain is quite small and we want wavefunction to be strongly concentrated around the nucleus, we use a hydrogen-like ion with $Z = 10$ to obtain a stronger Coulomb potential while retaining a one-electron system.

As 104 depends only on the distance r from the nucleus, the potential is radially symmetric. Hence, instead of solving problem on the squared domain $\Omega^2 = [0, 1]^2$ with n points per axis, we will choose a polar-coordinate-based mesh as it is more natural for Coulomb potential since the geometry of the potential is represented in terms of radial and angular coordinates. For simplicity will put the nucleus in the middle at $(x_0, y_0) = (0.5, 0.5)$ with 2 mesh parameters N_r and N_θ , denoting the numbers of radial and angular divisions, respectively, with $r \in [0, R]$ and $\theta \in [0, 2\pi)$. The idea of the mesh on the new domain can be visualised as in Fig. 26 where as we can see mesh consists of rings, where the space between the rings is subdivided into triangles. Also since we are analysing the nucleus we want more nodes in the middle, because the wavefunction varies rapidly near the nucleus and a finer mesh resolves the spatial behaviour more accurately, in addition the singularity of the original Coulomb potential at $r = 0$ is treated separately using the regularisation introduced below.

Because from the previous experiments we had only number of points per axis we will use them as radial divisions numbers and for the angular divisions we will use the algorithm that will keep the desired ratio $\frac{\text{angular element length}}{\text{radial element length}}$, in this project the target ratio is 1.3.

One of the main issues that arises from the first glance is the singularity when $r = 0$. This will be solved by using the regularised version of Coulomb potential 105:

$$V(r) = -\frac{Z}{r_{reg}}, \quad r_{reg} = r + \epsilon e^{-\frac{r}{\epsilon}} \quad (105)$$

This ensures that we avoid the division by 0 because when $r = 0$ we will have $V(0) = -\frac{Z}{\epsilon}$, because denominator would be finite, while being close to 0. For $r \gg \epsilon$ the exponential term becomes negligible and $r_{constr} \approx r$, so the constrained potential approaches the original Coulomb potential.

Another aspect that needed a change, comparing to the all previous computations, is the values that we are going to use as reference to compare obtained FEM numbers.

At first, the same separation of variables approach as for the quantum harmonic oscillator was taken, writing $\psi(r, \theta) =$

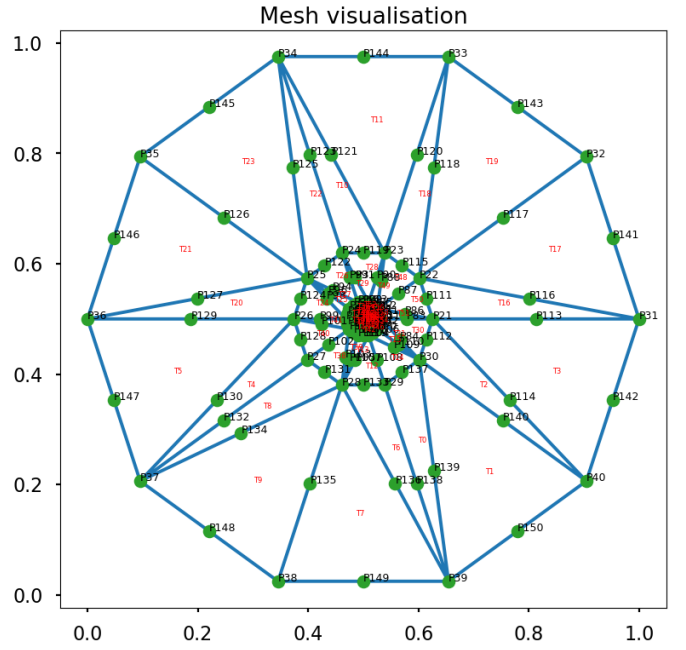


Fig. 26: The triangular mesh on the disk with P2 elements

$R(r)e^{im\theta}$ and substituting $u(r) = \sqrt{r}R(r)$ to eliminate the first derivative term, giving the standard radial form:

$$-u''(r) + \left[V(r) + \frac{m^2 - \frac{1}{4}}{r^2} \right] u(r) = Eu(r) \quad (106)$$

However, for the ground-state radial problem ($m = 0$), neither finite-difference discretisation nor a P1 weak form discretisation of this equation converged reliably under mesh refinement; both instead converged to a stable but incorrect value. The cause was traced to the substitution itself: for $m = 0$ the true solution $R(r)$ is smooth and generally nonzero at the origin (physically, $m = 0$ is the only angular-momentum state permitted nonzero amplitude at the nucleus), so $u(r) = \sqrt{r}R(r)$ behaves like $\sqrt{r}$ near $r = 0$ – a function with infinite slope at the origin. No polynomial basis, piecewise-linear (P1) or centred finite-difference, can represent this behaviour, regardless of resolution near $r = 0$.

The resolution was to abandon the substitution and solve directly for $R(r)$. Deriving the weak form of the untransformed radial equation with the correct polar measure $r dr$ shows that, for $m = 0$, the singular centrifugal term vanishes identically, leaving an entirely regular weak form:

$$\int_0^R R' \varphi' r dr + \int_0^R V(r) R \varphi r dr = E \int_0^R R \varphi r dr \quad (107)$$

with $R(r)$ free (rather than constrained to zero) at $r = 0$. A P1 discretisation of this direct formulation converged rapidly and matched the P2 FEM solution to within 0.02%, confirming that the FEM implementation was correct throughout and the earlier discrepancy originated entirely in the reference solver. Because of its simplicity and low computational cost, this

direct formulation was solved with $N = 5000$ P1 elements, verified sufficiently fine as a reference in VII-A.

1) *Results: eigenvalues:* Eigenvalues were obtained and compared with reference ones (important note, reference numbers are not real values that we get, these are the values that are incredibly close to the actual ones and obtained by the trusted method). The results are presented in Tab. XII for the first eigenvalues and in Tab. XIII for the degenerate pair.

TABLE XII: First eigenvalues errors with Coulomb potential

Points per axis	Interior DOFs	$\lambda_{1,h}$	$ \lambda_{1,ref} - \lambda_{1,h} $
5	131	-75.980390	2.943116
11	704	-78.781561	0.141944
21	2542	-78.905177	0.018328
41	9892	-78.920242	0.003263
81	38675	-78.922759	0.000746

TABLE XIII: Degenerate pair eigenvalues errors with Coulomb potential

Points per axis	Interior DOFs	$\max(\lambda_{2,h}, \lambda_{3,h})$	$ \lambda_{deg,ref} - \lambda_{deg,h} $
5	131	21.445939	6.244189
11	704	16.223882	1.022133
21	2542	15.470698	0.268949
41	9892	15.269349	0.067599
81	38675	15.219181	0.017431

As we can notice the errors are not as small as they were in the previous problems, but before making any conclusions it is a good practice to check the relative error from 79. For the first eigenvalue at the finest mesh $e_{rel} = \frac{0.000746}{-78.922759} \approx 9.45 \times 10^{-6}$. And for the degenerate pair $e_{rel} \approx 0.0011$. The first eigenvalue relative error matches the previous P2 element from the Laplace eigenproblem, however the degenerate pair has much higher error.

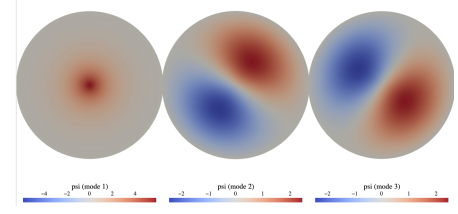
From this we can conclude that eigenvalues were converging towards specific limiting values, which provides numerical evidence that the implementation is behaving as expected.

2) *Results: eigenfunctions:* Similar to eigenvalues, eigenfunctions showed predictable behaviour with the ground-state wavefunction having the largest magnitude in the middle of the disk (near the nucleus), while the corresponding probability density is strongly concentrated around the centre of the disk. The two degenerate excited states exhibit nodal structure, with the regions of opposite sign separated by a nodal line as it is shown in Fig. 27a.

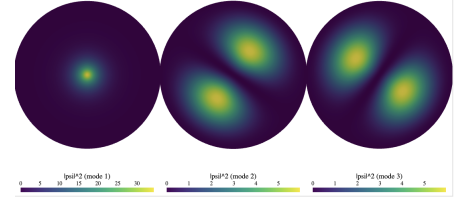
Moreover, we can notice in Fig. 27b that the probability density is concentrated around the same spots of the domain, similarly to the quantum harmonic oscillator. This qualitative similarity is expected because in both cases the potential confines the low-energy state around its minimum.

From [41] we know that two degenerate eigenfunctions are related by a 90 degrees rotation, as expected from the rotational symmetry of the Coulomb potential, and a corresponding relationship can be observed in Fig. 27a.

Finally, eigenfunction errors were decreasing as mesh becomes finer which is proved by the result in Table XIV.



(a) Eigenfunctions



(b) Probability density of the Coulomb potential

Fig. 27: Coulomb problem eigenfunctions and probability density

TABLE XIV: Ground state eigenfunction errors

Points per axis	L^2 -norm projection error	H^1 -norm projection error
5	0.045230	0.700917
11	0.001744	0.079745
21	0.000703	0.019350
41	0.000228	0.005964
81	0.000066	0.002125

The errors for the degenerate pair can be found in Table XV.

TABLE XV: Excited states (degenerate pair) eigenfunction errors

Points per axis	L^2 -norm projection error	H^1 -norm projection error
5	0.022764	0.535830
11	0.010916	0.213444
21	0.004405	0.083545
41	0.001348	0.029705
81	0.000379	0.010731

In both tables it is noticeable that the H^1 -norm projection error is noticeably higher than the corresponding L^2 -norm error. This is expected from the standard finite-element approximation theory rather than any limitation of the reference solution: for P2 elements, the H^1 error converges as $O(h^2)$ while the L^2 error converges as $O(h^3)$, one order faster, since the H^1 -norm measures errors in the derivatives of the eigenfunction, which a piecewise-quadratic approximation captures one order less accurately than the function values themselves. At any fixed mesh size h , this means the H^1 error is inherently larger than the L^2 error, independent of the reference accuracy. The reference solution's own contribution to this gap is in fact negligible: comparing the $N = 5000$ and $N = 40\,000$ -element P1 reference solutions directly gives a residual difference on the order of 10^{-7} , several orders of magnitude smaller than the reported H^1 errors themselves.

The consistent decrease of both errors under mesh refinement provides evidence of convergence, at the rates discussed in the following section.

3) *Convergence*: Notwithstanding good results in error analysis, the convergence analysis did not show the expected results. First of all lets look at Table XVI and Table XVII.

TABLE XVI: Eigenvalues convergence of the Coulomb potential

h	$\lambda_{1,h}$	rate of conv	$\max(\lambda_{2,h}, \lambda_{3,h})$	rate of conv
0.405071		NaN		NaN
0.246174		6.087635		3.633880
0.146306		3.933956		2.565867
0.079500		2.829338		2.264014
0.041601		2.278367		2.092796

TABLE XVII: Eigenfunctions convergence with Coulomb potential

h	Ground state		Degenerate pair	
	L^2	H^1	L^2	H^1
0.405071	NaN	NaN	NaN	NaN
0.246174	6.536669	4.364368	1.475737	1.848184
0.146306	1.746220	2.721565	1.743944	1.802654
0.079500	1.847717	1.929589	1.941339	1.695334
0.041601	1.921016	1.593131	1.959467	1.572191

The first noticeable feature comparing to the all previous results is that with the mesh refinement the convergence rates are decaying and approaching much lower value than was expected.

The main reason for this is that theoretical P2 convergence rates, namely $O(h^4)$ for eigenvalues, $O(h^3)$ for the L^2 eigenfunction error and $O(h^2)$ for the H^1 eigenfunction error, assume a shape-regular family of meshes. In particular, the aspect ratios of the elements must retain bounded independently of the mesh size h .

That is where the main difference lies, the polar mesh used for the Coulomb problem does not the condition stated above. At the centre of the disk, a single node is connected to all angular divisions, producing a fan of triangles. As the angular resolution is increased, the number of triangles meeting at this central node also increases. For this geometry, the worst-case aspect ratio of the fan grows approximately as $\frac{N_\theta}{2\pi}$, which was also confirmed against the measured mesh data up to rounding. Therefore, as N_θ increases during mesh refinement, the aspect ratio of the central elements becomes unbounded. This violates the shape-regularity assumption regardless of the radial grading or the chosen target aspect ratio.

Another reason for the decaying convergence rates is the interpretation of the global mesh size h . For a general triangulation, $h = \max_{K \in \Omega_h} \text{diam}(K)$. In previous examples, we used quasi-uniform meshes, for which the element sizes were comparable throughout the domain, so its global value provided a good representation of the overall mesh resolution. In the polar mesh for the Coulomb problem, however, the

elements near the centre are much smaller than those near the boundary, because the power-law radial distribution was used to cluster the nodes near the nucleus. Consequently, the global mesh value of h is determined by the largest elements and does not fully represent the much finer resolution near the nucleus.

These two reasons explain why the theoretical P2 convergence rates are not observed for the present mesh family. But it does not mean that P2 finite-element approximation failed, it means that this particular polar mesh family does not satisfy the geometric assumptions required by the standard convergence theory.

Improving the mesh topology so that the elements remain shape-regular while retaining increased resolution near the nucleus would require substantial changes to the mesh-generation procedure. This is therefore considered beyond the scope of the present project and could form the basis of a separate investigation.

XI. CONCLUSION

To conclude, all discussed problems have their own important physical meaning, however, because they belong to the one group of mathematical objects and can be formulated as elliptic eigenvalue problems of the form:

$$a(u, v) = \lambda b(u, v) \quad (108)$$

with potential changing the bilinear form, that is why algorithms to their solutions are quite identical.

Of course in this project all the problems we simplified with scaled versions and did not use the actual metrics, but they showed the general solution steps, that can be implemented as well for the scaled formulations and will produce more physically accurate numbers.

For partial differential equations finite-element method showed its strengths and applicability. It has many advantages, such as simple implementation, variety of approximations (P1, P2 etc elements), systematic convergence under appropriate mesh conditions and ability to build high quality visualisations and approximations. There are also some disadvantages, for example comparing to the finite-differences it is more complex to study, higher order convergence can be achieved with higher order elements that require more degrees of freedom which leads to increase in computational and implementation complexity.

Nevertheless, finite-element is a very strong mathematical tool that proved its effectiveness in both previous studies and this project.

REFERENCES

- [1] W. A. Strauss, *Partial Differential Equations*. Wiley Global Education, 2012
- [2] Arxiv.org, 2023. <https://arxiv.org/pdf/1903.11240>
- [3] W. Ding and Y. Wei, “Generalized Tensor Eigenvalue Problems,” *Theory and Computation of Tensors*, pp. 11-36, 2016, doi: 10.1016/b978-0-12-803953-3.50002-2
- [4] A. K. Saad, “An algorithm for the generalized matrix eigenvalue problem $Ax = \lambda Bx$,” ResearchGate. [Online]. Available: https://www.researchgate.net/publication/235012188_An_Algorithm_for_the_Generalized_Matrix_Eigenvalue_Problem_Ax_Lambda_Bx
- [5] “What is the difference between strong form and weak form? — ResearchGate,” ResearchGate, 2016. https://www.researchgate.net/post/What_is_the_difference_between_strong_form_and_weak_form
- [6] “Variational Formulation 2.1 Boundary Value problems Example 2.1.1 (One dim’1 problem).”, [Online]. Available: <https://mathsci.kaist.ac.kr/~dykwak/Courses/FEM765-17/variation-fem.pdf>
- [7] M. G. Larson and Fredrik Bengzon, *The Finite Element Method: Theory, Implementation, and Applications*. Springer Science & Business Media, 2013
- [8] GeeksforGeeks, “Bilinear Form,” GeeksforGeeks, Sept. 11, 2024. <https://www.geeksforgeeks.org/engineering-mathematics/bilinear-form/>
- [9] manavofficial577, “Bilinear Form Linear Algebra Notes,” Scribd, 2026. <https://www.scribd.com/document/931572146/Bilinear-Form-Linear-Algebra-Notes>
- [10] D. Grinberg, “Bilinear forms,” University of Minnesota. [Online]. Available: <https://www.cip.ifi.lmu.de/~grinberg/algebra/bilf.pdf>
- [11] “Lax-Milgram Theorem,” WolframResearch, 2026. <https://mathworld.wolfram.com/Lax-MilgramTheorem.html>
- [12] D. bounded, “Mathematics Stack Exchange,” Mathematics Stack Exchange, Mar. 28, 2023. <https://math.stackexchange.com/questions/4668104/determining-the-bilinear-form-a-is-bounded>
- [13] S. Shaik, “Compare Ritz to Galerkin,” Scribd, 2026. <https://www.scribd.com/document/314330469/Compare-Ritz-to-Galerkin>
- [14] “6 Eigenvalues of the Laplacian.” [Online]. Available: <https://web.stanford.edu/class/math220b/handouts/eigenvalues.pdf>
- [15] IEEE, “The Advantages of the Finite Element Method - IEEE Innovation at Work,” May 2019. [Online]. Available: <https://innovationatwork.ieee.org/the-advantages-of-fem/>
- [16] G. Strang and G. Fix, *An Analysis of the Finite Element Method*. Wellesley-Cambridge Press, 2018
- [17] I. Babuška and J. Osborn, “Eigenvalue problems,” in *Handbook of Numerical Analysis*, vol. II, Finite Element Methods (Part 1), P. G. Ciarlet and J. L. Lions, Eds. Amsterdam, Netherlands: North-Holland, 1991, pp. 641-787
- [18] G. Dusson and Y. Maday, “An overview of a posteriori error estimation and post-processing methods for nonlinear eigenvalue problems,” *Journal of Computational Physics*, vol. 491, p. 112352, July 2023, doi: 10.1016/j.jcp.2023.112352
- [19] I. Babuška and J. E. Osborn, “Finite element-Galerkin approximation of the eigenvalues and eigenvectors of selfadjoint problems,” *Mathematics of Computation*, vol. 52, no. 186, pp. 275-297, 1989, doi: 10.1090/s0025-5718-1989-0962210-8
- [20] GeeksforGeeks, “Triangulations Using Matplotlib,” GeeksforGeeks, Feb. 02, 2023. <https://www.geeksforgeeks.org/python/triangulations-using-matplotlib/>
- [21] J. Burkardt, “Computational Geometry Lab: FEM BASIS FUNCTIONS FOR A TRIANGLE,” 2018. [Online]. Available: https://people.math.sc.edu/Burkardt/classes/cg_2007/cg_lab_fem_basis_triangle.pdf
- [22] N. G. Gençer and I. O. Tanzer, “Forward problem solution of electromagnetic source imaging using a new BEM formulation with high-order elements,” *Physics in Medicine and Biology*, vol. 44, no. 9, pp. 2275-2287, Aug. 1999, doi: 10.1088/0031-9155/44/9/314
- [23] M. Igel, “Finite elements: Basis functions,” Ludwig Maximilian University of Munich. [Online]. Available: https://www.geophysik.uni-muenchen.de/~igel/Lectures/NMG/08_finite_elements_basisfunctions.pdf
- [24] D. Xu, J. Stare, and A. L. Cooksy, “Solving the vibrational Schrödinger equation on an arbitrary multidimensional potential energy surface by the finite element method,” *Computer Physics Communications*, vol. 180, no. 11, pp. 2079-2094, June 2009, doi: 10.1016/j.cpc.2009.06.010
- [25] Hammer, Preston C., Marlowe, O. J., and Stroud, A. H. Numerical integration over simplexes and cones, *Mathematical Tables and Other Aids to Computation* 10(55), 130-137, 1956. [DOI: 10.2307/2002483]
- [26] D. A. Dunavant, “High degree efficient symmetrical Gaussian quadrature rules for the triangle,” *International Journal for Numerical Methods in Engineering*, vol. 21, no. 6, pp. 1129-1148, June 1985, doi: 10.1002/nme.1620210612
- [27] J. Burkardt, “triangle_dunavant_rule.cpp,” Triangle Dunavant Rule, [C++ source code]. [Online]. Available: https://people.math.sc.edu/Burkardt/cpp_src/triangle_dunavant_rule/triangle_dunavant_rule.cpp
- [28] “Assembly — FEM Tutorial 0.1.0 documentation,” Uni-erlangen.de, 2017. <https://ww8central.ww.uni-erlangen.de/courses/fem101/fem/assembly.html>
- [29] “Chapter 25,” iFEM: Introduction to Finite Element Methods. [Online]. Available: <https://quickfem.com/wp-content/uploads/IFEM.Ch25.pdf>
- [30] “Introduction to Finite Elements,” Inrialpes.fr, 2026. <https://imagine.inrialpes.fr/people/Francois.Faure/htmlCourses/FiniteElements.html>
- [31] Rakesh Arugonda, “Dirichlet Boundary Condition,” Scribd, 2026. <https://www.scribd.com/doc/172742570/Dirichlet-Boundary-Condition>
- [32] “7. Dirichlet boundary conditions — Finite element course 2024.0 documentation,” 2024. [Online]. Available: https://finite-element.github.io/7_boundary_conditions.html
- [33] P. G. Ciarlet, *The Finite Element Method for Elliptic Problems*. SIAM, 2002
- [34] “scipy.linalg.eigh — SciPy v1.10.1 Manual.” [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.eigh.html>
- [35] R. Courant and D. Hilbert, *Methods of Mathematical Physics, Volume I*
- [36] “Degenerate eigenvalues,” Bohrium SciencePedia. [Online]. Available: https://www.bohrium.com/en/sciencepedia/feynman/keyword/degenerate_eigenvalues
- [37] “Understanding Convergence Rates — Hermes Tutorial Guide,” Hpfem.org, 2026. https://hpfem.org/wp-content/uploads/doc-web/doc-tutorial/src/hermes2d/D-adaptivity/intro-2.html?utm_source=chatgpt.com
- [38] “2.4. Particle in a box with infinite-potential walls — Introduction to Quantum Physics,” York.ac.uk, 2019. https://www-users.york.ac.uk/~pjd113/notebooks/2_4_infinite_square_well.html
- [39] “7.6: The Quantum Harmonic Oscillator,” Physics LibreTexts, Nov. 01, 2016. [https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_\(OpenStax\)/University_Physics_III_-_Optics_and_Modern_Physics_\(OpenStax\)/07%3A_Quantum_Mechanics/7.06%3A_The_Quantum_Harmonic_Oscillator](https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_(OpenStax)/University_Physics_III_-_Optics_and_Modern_Physics_(OpenStax)/07%3A_Quantum_Mechanics/7.06%3A_The_Quantum_Harmonic_Oscillator)
- [40] S. Fraser, “Schrödinger equation for the nuclear potential,” NUCS 342: Introduction to Nuclear Science, Simon Fraser University, Jan. 24, 2011. [Online]. Available: <https://web-docs.gsi.de/~wolle/TELEKOLLEG/KERN/LECTURE/Fraser/L6.pdf>
- [41] “4.5 The Hydrogen Atom.” [Online]. Available: https://ocw.mit.edu/courses/6-974-fundamentals-of-photonics-quantum-electronics-spring-2006/8a3eb732190cc7fc2520fa122bed8dcd_hydrogen_atom.pdf

APPENDIX A
PROOF OF COERCIVITY

To prove that $a(v, v) \geq \alpha \|v\|_{H^1}^2$ for some $\alpha > 0$, evaluate the bilinear form at identical arguments:

$$\begin{aligned} a(v, v) &= \int_{\Omega} \nabla v \cdot \nabla v \, dx \\ &= \int_{\Omega} |\nabla v|^2 \, dx \\ &= \|\nabla v\|_{L^2}^2 \end{aligned} \quad (109)$$

Because Ω is a bounded domain, the Poincaré inequality (Appendix C) asserts that there exists a positive constant C_{Ω} depending only on the geometry of Ω such that:

$$\|v\|_{L^2}^2 \leq C_{\Omega}^2 \|\nabla v\|_{L^2}^2 \quad \forall v \in H_0^1(\Omega) \quad (110)$$

Relate 110 to the full H^1 -norm $(\|v\|_{H^1})^2 = \|v\|_{L^2}^2 + \|\nabla v\|_{L^2}^2$ gives:

$$\begin{aligned} \|v\|_{H^1}^2 &= \|v\|_{L^2}^2 + \|\nabla v\|_{L^2}^2 \\ &\leq C_{\Omega}^2 \|\nabla v\|_{L^2}^2 + \|\nabla v\|_{L^2}^2 \\ &= (C_{\Omega}^2 + 1) \|\nabla v\|_{L^2}^2 \end{aligned} \quad (111)$$

Isolating the gradient term gives:

$$\|\nabla v\|_{L^2}^2 \geq \frac{1}{C_{\Omega}^2 + 1} \|v\|_{H^1}^2 \quad (112)$$

Substituting 112 back into the 109 completes the proof of coercivity:

$$a(v, v) \geq \alpha \|v\|_{H^1}^2 \quad \text{where } \alpha = \frac{1}{C_{\Omega}^2 + 1} > 0 \quad (113)$$

APPENDIX B
PROOF OF BOUND LINEAR FUNCTIONAL

To establish the continuity of $L(v)$, we must show that $|L(v)| \leq C_L \|v\|_{H^1}$ for a constant $C_L > 0$.

To do so we apply the Cauchy-Schwarz inequality (Appendix D) in $L^2(\Omega)$ to the definition of the linear functional:

$$\begin{aligned} |L(v)| &= \left| \int_{\Omega} f v \, dx \right| \\ &\leq \|f\|_{L^2} \|v\|_{L^2} \end{aligned} \quad (114)$$

By the definition of the H^1 -norm, it is clear that $\|v\|_{L^2} \leq \|v\|_{H^1}$ because $\|v\|_{H^1} = \sqrt{\|v\|_{L^2}^2 + \|\nabla v\|_{L^2}^2}$. Substituting this property into the upper bound gives:

$$|L(v)| \leq \|f\|_{L^2} \|v\|_{H^1} \quad (115)$$

Setting $C_L = \|f\|_{L^2}$ satisfies the continuity condition:

$$|L(v)| \leq C_L \|v\|_{H^1} \quad (116)$$

Since $f \in L^2(\Omega)$, the value C_L is a finite constant independent of the choice of v , confirming that L is a bounded, continuous linear operator on $H_0^1(\Omega)$.

APPENDIX C
POINCARÉ INEQUALITY

From [7].

Let $\Omega \subset \mathbb{R}^2$ be a bounded domain. Then, there is constant $C = C(\Omega)$, such that for any $v \in V_0$,

$$\|v\|_{L^2(\Omega)} \leq C \|\nabla v\|_{L^2(\Omega)} \quad (117)$$

Proof.

Let ϕ be a function satisfying $-\Delta\phi = 1$ in Ω with $\sup_{x \in \Omega} |\nabla\phi(x)| < C$. Such a function exists if the boundary of Ω is sufficiently smooth.

Multiplying $1 = -\Delta\phi$ with v^2 , and integrating by parts using Green's formula 10 we have

$$\begin{aligned} \int_{\Omega} v^2 \, dx &= - \int_{\Omega} v^2 \Delta\phi \, dx \\ &= - \int_{\partial\Omega} v^2 n \cdot \nabla\phi \, ds \\ &\quad + \int_{\Omega} 2v \nabla v \cdot \nabla\phi \, dx \\ &= \int_{\Omega} 2v \nabla v \cdot \nabla\phi \, dx \end{aligned} \quad (118)$$

since $v = 0$ on $\partial\Omega$. Using the Cauchy-Schwarz inequality from Appendix D we further have

$$\begin{aligned} \|v\|_{L^2(\Omega)}^2 &= \int_{\Omega} v^2 \, dx \\ &= \int_{\Omega} 2v \nabla v \cdot \nabla\phi \, dx \\ &\leq 2 \max |\nabla\phi| \|v\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \end{aligned} \quad (119)$$

Finally, we set $C = 2 \max |\nabla\phi|$, and divide by $\|v\|_{L^2(\Omega)}$.

The above prove holds if v is zero on the part of the boundary or if the average $|\Omega|^{-1} \int_{\Omega} v \, dx$ of v is zero.

APPENDIX D
CAUCHY-SCHWARZ INEQUALITY

$$\int_{\Omega} v w \, dx \leq \|v\|_{L^2(\Omega)} \|w\|_{L^2(\Omega)} \quad (120)$$